Page No. :
Date :

~~RECURSION~~

—GRAPHS—

Graphs Pattern :-

01. DFS - Recursion / Stack
Exploring all paths, connected components, islands, topological sort, cycles
200 - No. of Islands
130 - Surrounded Regions
547 - No. of Provinces

02. BFS - Queue - finding Shortest path (Unweighted), levels of traversal, spreading processes.
994 - Rotting Oranges
1091 - Shortest Path In Binary Mat
127 - Word Ladder

03. Topological Sort (Kahn's Algo or DFS)
Dealing with DAGs and ordering tasks with prerequisites
207 - Course Schedule
210 - Course Schedule II
269 - Alien Dictionary (Hard)

04. Cycle Detection - You need to detect if cycles exist (in directed / undirected graphs)
785 - Is Graph Bipartite?
261 - Graph Valid Tree
207 - Course Schedule

05. Union - Find / Disjoint Set Union (DSU) - You need to group components or detect cycles efficiently
684 - Redundant Connection
1319 - No. of Ops to make net. conn.
1202 - Smallest String with Swaps

06. Dijkstra's Algorithm (Min-Heap) - Weighted graph & you need the shortest path
743 - Network Delay Time
1631 - Path with Minimum Effort
787 - Cheapest Flights within K Stops

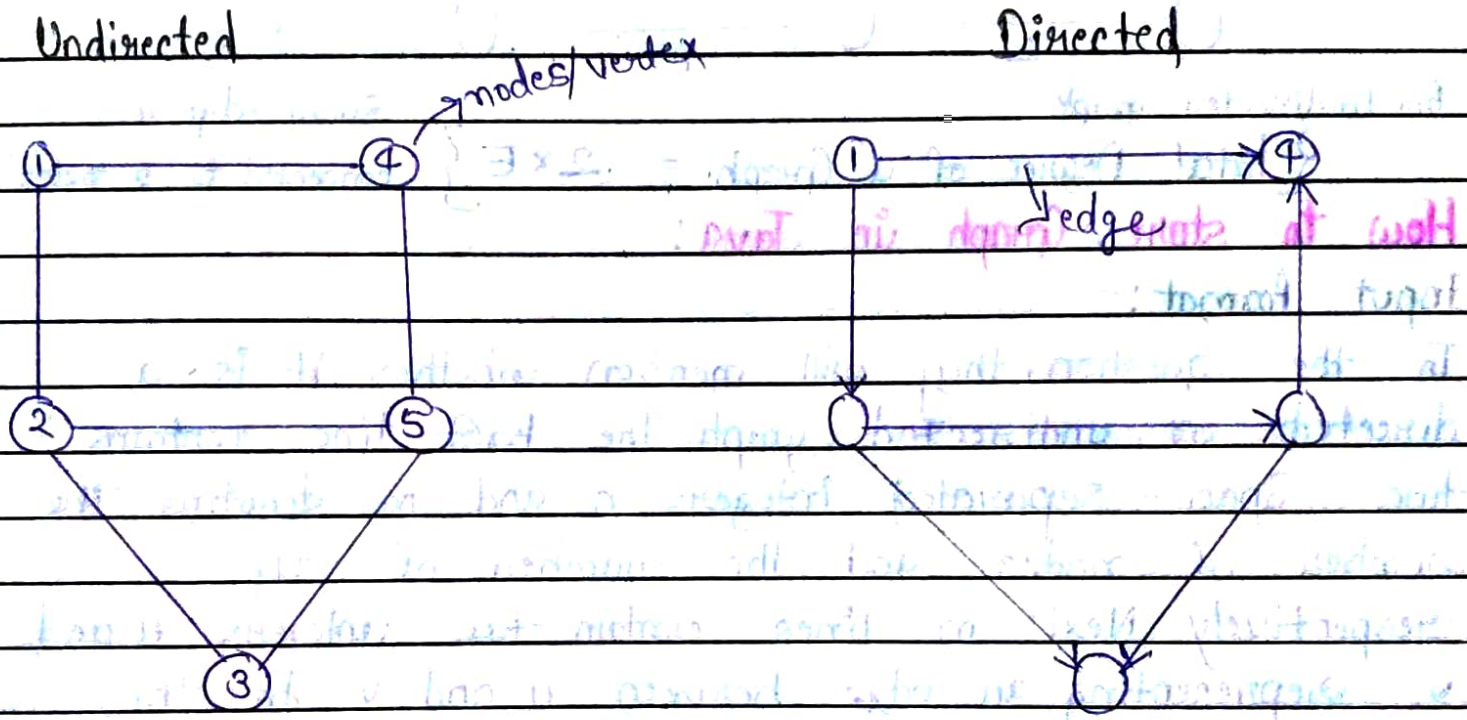
Page No. :
Date :

01. Number of Provinces
02. No. of Islands / No. of Connected Components in a Matrix
03. Flood Fill Algorithm
04. Rotten Oranges - direction +
05. Detect a Cycle in an Undirected Graph using BFS
06. Detect a Cycle in an Undirected Graph using DFS
07. Distance of nearest cell having 1 / 0-1 / matrix
08. Surrounded Regions / Replace O's with X's
09. Number of Enclaves
10. Number of Distinct Islands
11. Bipartite Graph (BFS)
12. Bipartite Graph (DFS)
13. Detect a cycle in a Directed Graph (DFS)
14. Find Eventual Safe States (DFS)
15. Topological Sort Algo. (DFS)
16. Topological Sorting (Kahn's Algo / BFS)
17. Detect a Cycle in Directed Graph (Topological Sort / Kahn's Algo)
18. Course Schedule I and II
19. Find Eventual Safe State. BFS
20. Alien Dictionary
21. Shortest Path in directed acyclic Graph
22. Shortest Path in Undirected graph with Unit weights
23. Word - Ladder I
24. Word Ladder - II
25. Dijkstra's Algorithm
26. Print Shortest Path
27. Shortest Dist. in a Binary Maze
28. Path With Minimum Effort
29. Cheapest Flights within K stops
30. min. Multiplication to Reach End
31. No. of ways to arrive at destination
32. Bellman Ford Algorithm

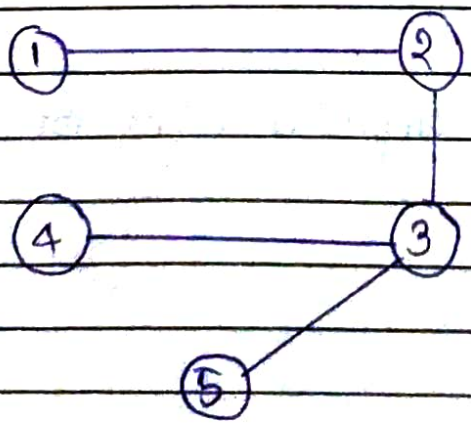
Page No. :
Date :

Graph Introduction :- A graph is a non-linear data structure consisting of nodes that have data and are connected to other nodes through edges.

Types of Graph :-



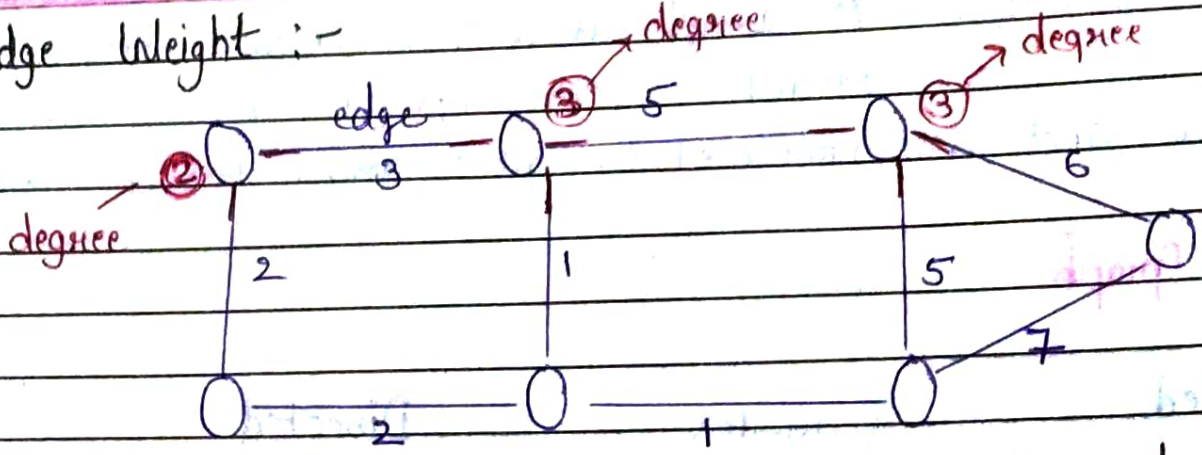
Path :- Contain a lot of nodes and each of them are reachable.



1 2 3 5 → path

Page No. :
Date :

Edge Weight :-



For Undirected graph :-

{Total Degree of a Graph = 2 × E} Every edge is connected to 2 nodes.

How to store Graph in Java :-

Input format :-

In the question, they will mention whether it is a directed or undirected graph. The first line contains two space-separated integers n and m denoting the number of nodes and the number of edges respectively. Next m lines contain two integers u and v representing an edge between u and v. In the case of an undirected graph if there is an edge between u and v, it means there is an edge between v and u as well. Now the question arises if there is any boundation on the number of edges, i.e. the value of m ? The answer is No, if we add more edges, then the value of m will increase.

The two most commonly used representations for graphs are :-

01. Adjacency Matrix
02. Adjacency Lists.

Page No. :
Date :

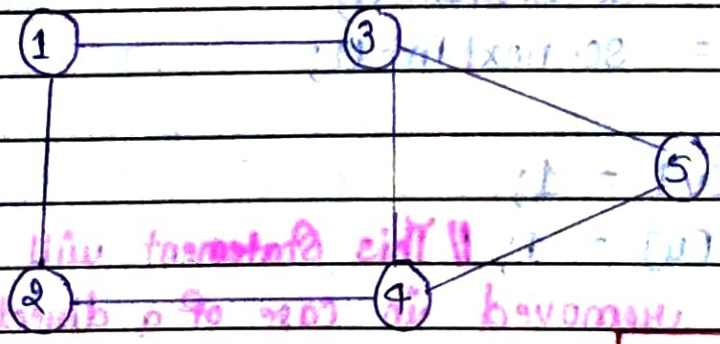
01. Adjacency Matrix :-

An adjacency matrix of a graph is a two-dimensional array of size $n \times n$, where n is the number of nodes in the graph, with the property that $a[i][j] = 1$ if the edge (v_i, v_j) is in the set of edges, and $a[i][j] = 0$ if there is no such edge.

Input :-

(5) (6)
1 2
1 3
2 4
3 4
3 5
4 5

Explanation :-
Number of nodes, $n = 5$
Number of edges, $m = 6$
Next m lines represent the edges.



Undirected graph

Space = N x N

	0	1	2	3	4	5
0						
1			1	1		
2		1			1	
3		1			1	1
4			1	1		1
5				1	1	

– Adjacency Matrix –

Page No. :
Date :

Adjacency matrix for undirected graph -

```
import java.util.Scanner;

public class Main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt(); // Number of nodes
        int m = sc.nextInt(); // Number of edges

        // Adjacency matrix for undirected graph
        // Time complexity : O(n)
        int adj[][] = new int[n+1][n+1];

        for (int i = 0; i<m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            adj[u][v] = 1;
            adj[v][u] = 1; // This Statement will be
                           removed in case of a directed graph
        }
        sc.close();
    }
}
```

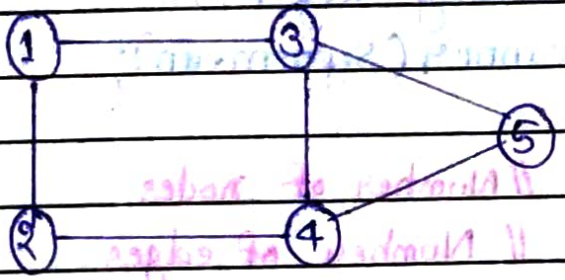
For weighted Graph:-

```
adj[u][v] = wt
```

Page No. :
Date :

⇒ Adjacency Lists - This is a node-based representation. In this representation, we associate with each node a list of nodes adjacent to it. Normally an array is used to store the nodes.

Consider the example of the following undirected graph-



motive :- define which nodes are neighbour of a perticular node.

To create an adjacency list, we will create an array of size $n+1$ where n is the number of nodes. This array will contain a list.

Space = O(2E)

0	→ {}	1 2
1	→ {2,3}	1 3
2	→ {1,4,}	2 4
3	→ {1,4,5}	3 4
4	→ {2,3,5}	3 5
5	→ {3,4}.	4 5

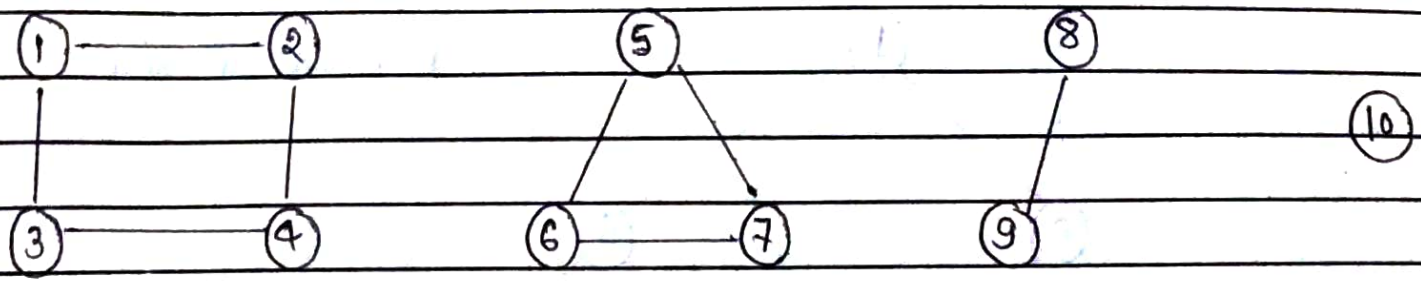
Page No. :
Date :

Adjacency list for undirected graph:-

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt(); // Number of nodes
        int m = sc.nextInt(); // Number of edges

        // Adjacency list for undirected graph
        // Time complexity : O(2E)
        ArrayList<ArrayList<Integer>> adj = new ArrayList<>();

        // Initialize the adjacency list (n+1) arraylist
        for(int i = 0; i <= n; i++) {
            adj.add(new ArrayList<>());
        }

        Input Edges
        for(int i = 0; i < m; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            adj.get(u).add(v);
            adj.get(v).add(u); // Remove it for directed graph
        }

        sc.close();
    }
}
```

Page No. :
Date :

⇒ Connected Components in Graphs – N = 10, M = 8



4 Components

12
13
24
34
56

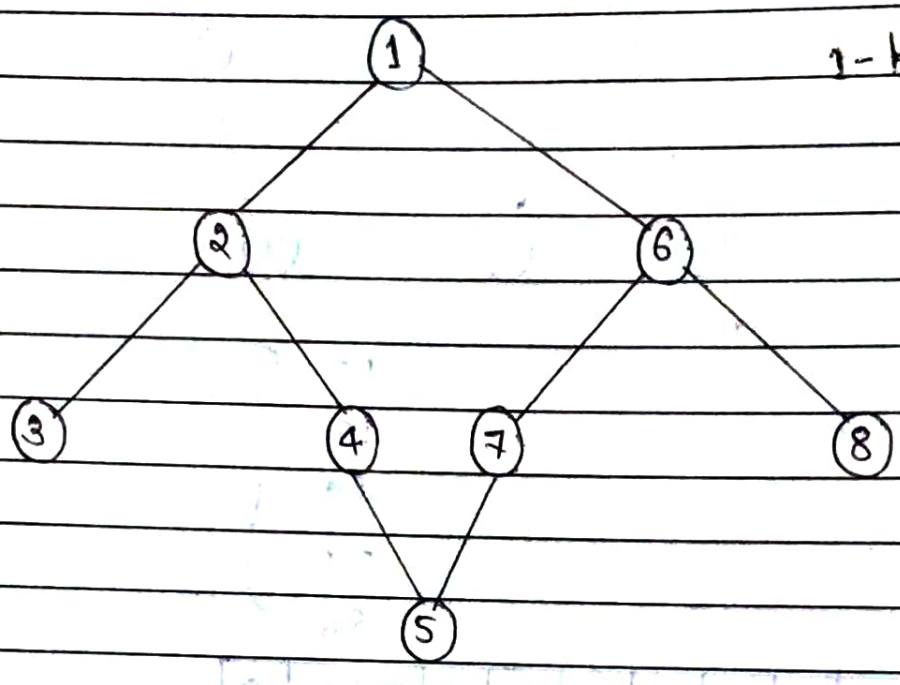
visited Array :-

0	0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9	10

```
for ( i = 1 to 10) {
    if (!visited[i]) {
        traversal(i);
    }
}
```

Page No. :
Date :

⇒ Breadth - First Search (BFS) :-



1-based indexing

Starting Node :- 1

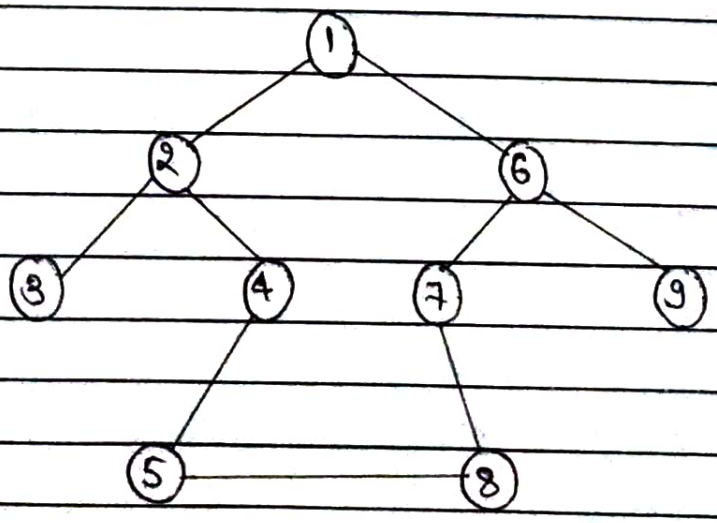
BFS - 1 2 6 3 4 7 8 5

0 1 2 3

starting Node - 6

BFS - 6 1 7 8 2 5 3 4

0 1 2 3



Page No. :
Date :

vis =

0	1	2	3	4	5	6	7	8	9
0	1	0	0	0	0	0	0	0	0

0 → {}
1 → {2, 6}
2 → {1, 3, 4}
3 → {2}
4 → {2, 5}
5 → {4, 8}
6 → {1, 7, 9}
7 → {6}
8 → {5, 7}
9 → {6}

Adjacency List

Queue (FIFO)

5
9
7
4
3
6
2
1

BFS — 1 2 6

{Who are your neighbours}

— BFS —

TC - O(N) + O(2E)

SC - O(3N)

Page No. :
Date :

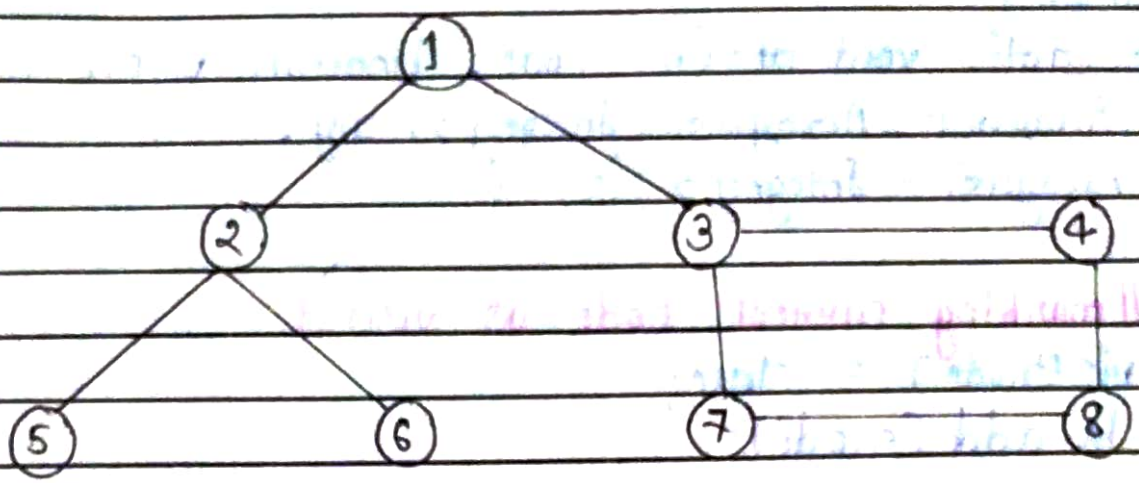
BFS Code :-

```
import java.util.*;
class Solution {
    public ArrayList<Integer> bfsOfGraph(int V,
        ArrayList<ArrayList<Integer>> adj) {
    // in this neighbours are already stored
        ArrayList<Integer> bfs = new ArrayList<>();
        boolean vis[] = new boolean[V];
        Queue<Integer> q = new LinkedList<>();

        q.add(0);
        vis[0] = true;

        while (!q.isEmpty()) {
            Integer node = q.poll();
            bfs.add(node);

    // Visiting neighbours
            for (Integer it : adj.get(node)) {
                if (vis[it] == false) {
                    vis[it] = true;
                    q.add(it);
                }
            }
        }
        return bfs;
    }
}
```

V → No. of Nodes

Page No. :
Date :

⇒ Depth First Search (DFS) :-



1 → {2, 3}
2 → {1, 5, 6}
3 → {1, 4, 7}
4 → {3, 8}
5 → {2}
6 → {2}
7 → {3, 8}
8 → {4, 7}

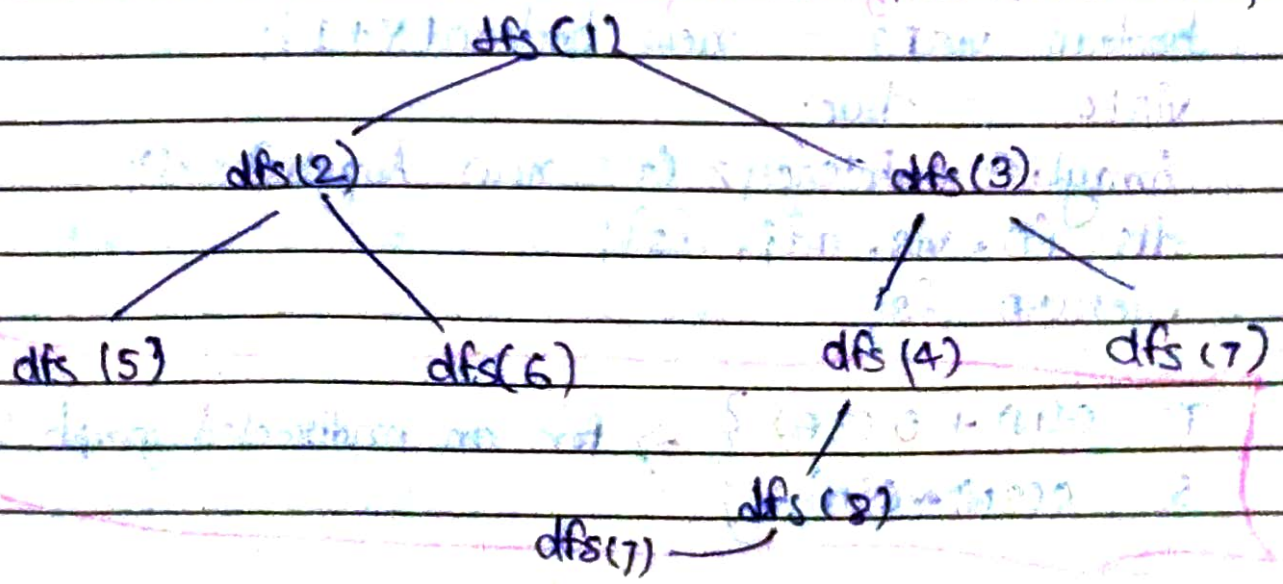
Starting Node :- 1
DFS :- 1 2 5 6 3 7 8 4

Starting Node :- 3
DFS :- 3 4 8 7 1 2 5 6

vis =

0	1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1	1

Recursion used here.



Page No. :
Date :

DFS Code:-

```
import java.util.*;
class Solution {
    public static void dfs(int node, boolean vis[],
        ArrayList<ArrayList<Integer>> adj,
        ArrayList<Integer> ls) {

        // marking current node as visited
        vis[node] = true;
        ls.add(node);

        // getting neighbour nodes
        for (Integer it : adj.get(node)) {
            if (vis[it] == false) {
                dfs(it, vis, adj, ls);
            }
        }
    }
}
```

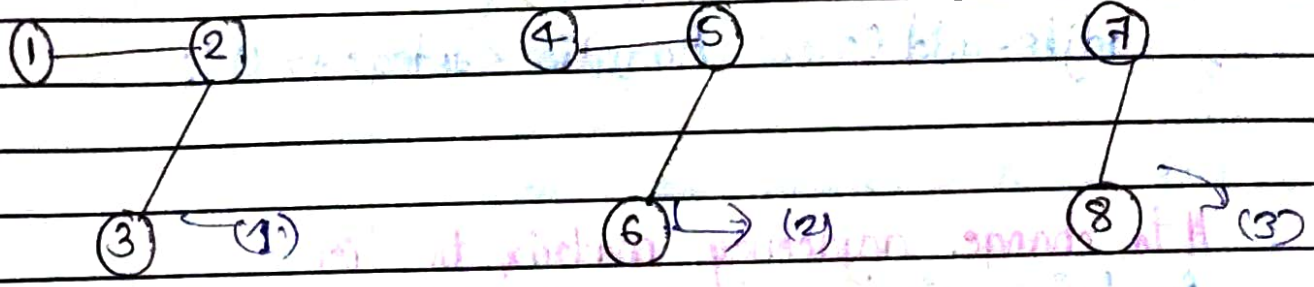
TC- O(N) + O(E)
SC- O(3N) ~ O(N)
⇒ for a directed graph...

```
// Function to return a list containing the DFS traversal of the graph
public ArrayList<Integer> dfsOfGraph(int V,
    ArrayList<ArrayList<Integer>> adj) {
    // boolean Array to keep track of visited vertices
    boolean vis[] = new boolean[V+1];
    vis[0] = true;
    ArrayList<Integer> ls = new ArrayList<>();
    dfs(0, vis, adj, ls);
    return ls;
}
```

TC- O(N) + O(2E)
SC- O(3N) ~ O(N)
⇒ for an undirected graph.

Page No. :
Date :

Q. Number of Provinces :-



Ans = 3

```
private static void dfs(int node, ArrayList<ArrayList<Integer>>
        adjls, int vis[]) {

    vis[node] = 1;

    for( Integer it : adjls.get(node)) {
        if ( vis[it] == 0) {
            dfs( it, adjls, vis);
        }
    }
}

static int numProvinces(ArrayList<ArrayList<Integer>>
        adj, int V ) {

    ArrayList<ArrayList<Integer>> adjls = new
            ArrayList<ArrayList<Integer>>();
```

Date :

```
for (int i=0; i<V; i++) {
    adjls.add(new ArrayList<Integer>());
}

// to change adjacency matrix to list
for (int i=0; i<V; i++) {
    for (int j=0; j<V; j++) {
        //self nodes are not considered
        if (adj.get(i).get(j) == 1 && i != j) {
            addls.get(i).add(j);
            addls.get(j).add(i);
        }
    }
}

int vis[] = new int[V];
int cnt = 0;

for (int i=0; i<V; i++) {
    if (vis[i] == 0) {
        cnt++;
        dfs(i, adjls, vis);
    }
}
return cnt;
}
```

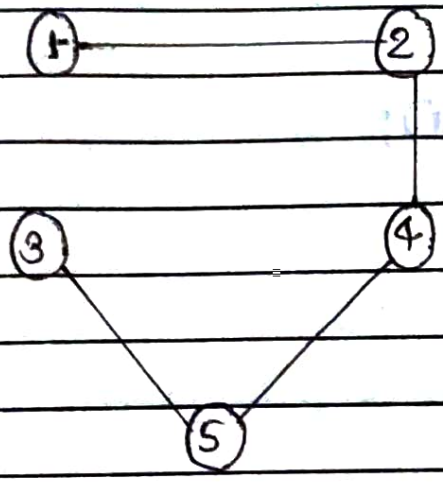
TC - O(N) + O(V + 2E)

SC - O(N) + O(N)

Page No. :
Date :

Degree

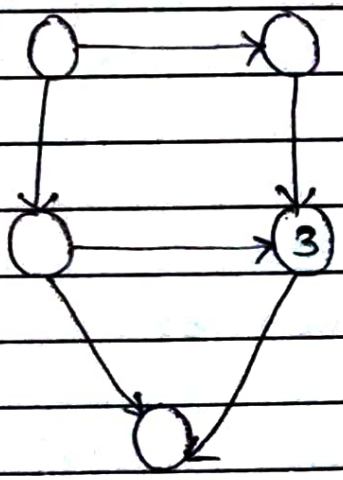
Undirected Graph:-



Degree = 2 × E

{Edges are connected to 2 nodes}

Directed Graph:-



Indegree (3) = 2

Outdegree (3) = 1

Indegree: Incoming Edges
Outdegree: Outgoing Edges

Page No. :
Date :

// ADJACENCY MATRIX

```
public static void main(String args[]){
    int n = 3, m = 3;
    int adj[][] = new int[n+1][n+1];

    adj[1][2] = 1;
    adj[2][1] = 1;

    adj[2][3] = 1;
    adj[3][2] = 1;

    adj[1][3] = 1;
    adj[3][1] = 1;

    adj[4][5] = 1;
    adj[5][4] = 1;

}
```

Page No. :
Date :

Adjacency List :-

```
public static void main(String args[]) {
    int n = 3,  m = 3;
    ArrayList<ArrayList<Integer>> adj = new ArrayList<ArrayList<
                                        Integer>>();
    //n+1
    for( int i=0; i<=n; i++ ) {
        adj.add(new ArrayList<Integer>());
    }

    // edge 1---2
    adj.get(1).add(2);
    adj.get(2).add(1);
    adj.get(u).add(v);
    adj.get(v).add(u);
    adj.get(2).add(3);    // edge 2---3
    adj.get(3).add(2);

    //edge 1---3
    adj.get(1).add(3);
    adj.get(3).add(1);

    for(int i=1; i<n; i++) {
        for(int j=0; j<adj.get(i).size(); j++) {
            Syso(adj.get(i).get(j) + " ");
        }
        Syso();
    }

}
```

Page No. :
Date :

How to store Weighted Graph ?

Adjacency matrix

① Undirected graph / Directed :- In place of storing 1 in the Adjacency matrix store weight of that perticular edge.

```
adj[u][v] = wt
```

② Adjacency List - Store with weight in pairs.

{1} ⟶ {{2,1}, {3,3}, {4,2}}

{3} ⟶ {{1,2}, {4,5}}

Page No. :
Date :

Dry Run of No. of Provinces -

```
class Solution {
    private void dfs(int[][] isConnected, boolean[] vis, int i) {
        vis[i] = true;

        for (int j = 0; j < isConnected.length; j++) {
            if (isConnected[i][j] == 1 &&
                !vis[j]) {

                dfs(isConnected, vis, j);
            }
        }
    }

    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length;
        boolean vis[] = new boolean[n];

        int cnt = 0;
        for (int i = 0; i < n; i++) {
            if (!vis[i]) {
                dfs(isConnected, vis, i);
                cnt++;
            }
        }

        return cnt;
    }
```

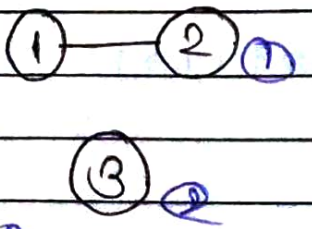
TC - O(V + 2E) + O(N²)
SC - O(N) + O(N)

Page No. :
Date :

Dry Run of No. of Provinces –

Input :- isConnected = [[1,1,0] [1,1,0] [0,0,1]]

Output = 2



n = 3 (length)

cnt = 0

vis = | F | F | F | F |
0 1 2 3

(1) i = 0

vis(0) = false

dfs (isConnected, vis, i(0))

cnt = 1

(2) dfs (isConnected, vis, 0)

vis (0) = true | T | F | F | |
0 1 2

for (j = 0 ⟶ n)

isConnected (i][j) == 1 &&
! vis[j)

dfs (isConnected, vis, j)

Page No. :
Date :

Dry Run of No. of Provinces —

③ vis [0] = True

T	F	F
0	1	2

④ j = 1

isConnect[0][1] == 1 && vis[1] = False

dfs (isConnected, vis, j).

⑤ vis[1] = True

T	T	F
0	1	2

⑥ j = 2

isConnected[0][2] == 0

⇒ ~~cnt~~ i = 1

dfs (isConnected, vis, 1);

cnt = 2

Page No. :
Date :

02. Number of Islands -/ No. of Connected Components in a Matrix -

direction - ✱ {But direction → 4 in Leetcode}

Given Grid of size n x m

0 → water 1 → land

	0	1	2	3
0	0	1	1	0
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	1	1	0	1

⇒ ans = 2.

⇒ Dry Run :- Create a replica of given matrix for visited array.

(1)

BFS :-

	0	1	2	3
0		✓	✓	
1		✓	✓	
2				
3				
4				

vis[][]

starting → {0,1} ✓

	0	1	2	3	
0	0	1	1	0	(given)
1	0	1	1	0	
2	0	0	1	0	
3	0	0	0	0	
4	1	1	0	1	

{1,1}
{1,2}
{0,2}
~~{0,1}~~

Queue

Page No. :
Date :

(2)

	0	1	2	3	
0		✓	✓		—
1		✓	✓		—
2					—
3					—
4					—

vis[][]

BFS :-

	0	1	2	3	4	
0	0	1	1	0		—
1	0	1	1	0		—
2	0	0	1	0		—
3	0	0	0	0		—
4	1	1	0	1		—

{0,2} ✓

{0,1} ←	
{1,1}	Already visited
{1,2}	Already visited
~~{0,2}~~	

Queue

(3)

	0	1	2	3
0		✓	✓	
1		✓	✓	
2			✓	
3				
4				

vis[][]

{0,1} → Already visited
{2,2} → new one
{1,1} → Already visited
{0,2} → BFS :-

	0	1	2	3
0	0	1	1	0
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	1	1	0	1

{1,2} ✓

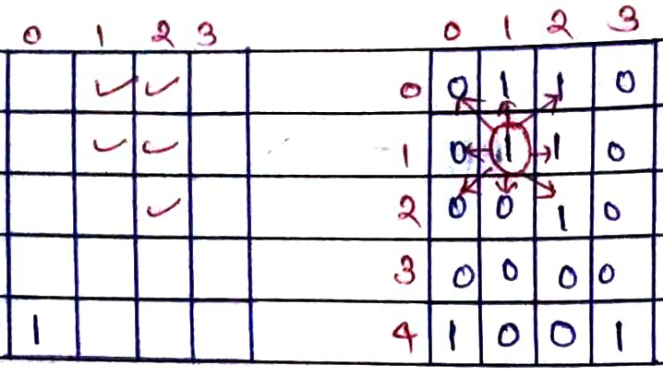
←
←
{2,2}
{1,1}
{1,2}

Queue

Page No. :
Date :

④ {1, 1}

BFS :-



vis[][]

given mat

Queue

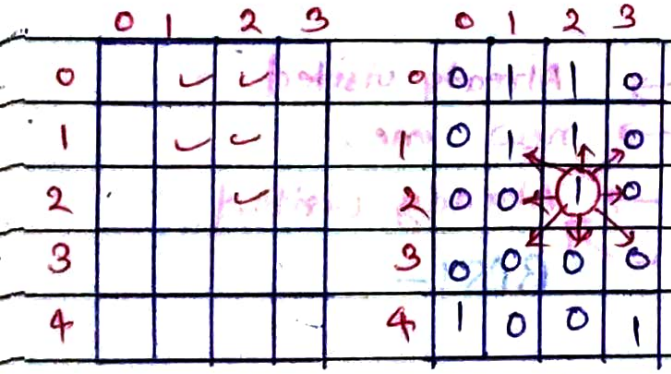
Queue
{2,2}
~~{1,1}~~

- {0, 1} → Already visited
- {0, 2} → —"— —"—
- {1, 2} → —"— —"—
- {2, 2} → —"— —"—

Remaining are water.

⑤ {2, 2}

BFS :-



vis[][]

given mat

Queue

Queue
~~{2,2}~~

- {1, 2} → Already visited
- {1, 1} → —"—
- Remaining are water
- Queue is Empty Now —
  Starting point will change →

BFS

Page No. :
Date :

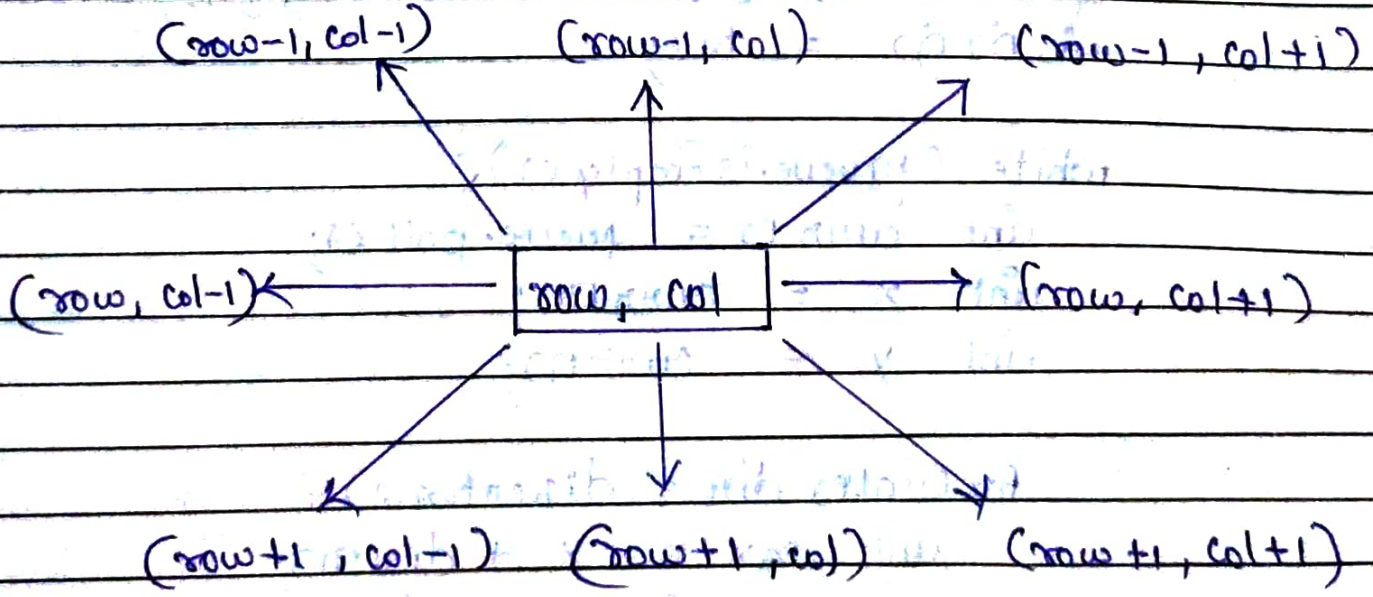
~~(6) {4,0}~~

```
public int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) {
        return 0;
    }
    int m = grid.length;
    int n = grid[0].length;
    int numIslands = 0;

    boolean[][] vis = new boolean[m][n];
    int[][] directions = {{1,0},{-1,0},{0,1},{0,-1}};
    Queue<int[]> queue = new ArrayDeque<>();

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (grid[i][j] == '1' && !vis[i][j]) {
                numIslands++;
                queue.offer(new int[]{i, j});
                vis[i][j] = true;

                while (!queue.isEmpty()) {
                    int[] curr = queue.poll();
                    int x = curr[0];
                    int y = curr[1];

                    for (int[] dir : directions) {
                        int nx = x + dir[0];
                        int ny = y + dir[1];
```

```
if (nx >= 0 && nx < m &&
    ny >= 0 && ny < n &&
    grid[nx][ny] == '1' &&
    !vis[nx][ny]){

    queue.offer(new int[]{nx, ny});
    vis[nx][ny] = true;
}
}
}
}
}
}
return numIslands;
}
```

TC - O(n * m)

SC - O(n * m) + O(n * M)
(visited) (queue)



Page No. :
Date :

Code:- ~~// BFS~~ // DFS

```
public int numIslands(char[][] grid) {
    if (grid == null || grid.length == 0) {
        return 0;
    }
    boolean[][] vis = new boolean[grid.length][grid[0].length];
    int numIslandss = 0;
    for (int i=0; i< grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1' && !vis[i][j]) {
                numIslandss++;
                dfs(grid, vis, i, j);
            }
        }
    }
    return numIslandss;
}
                                    boolean[][] vis
private void dfs(char[][] grid, int i, int j) {
    if (i<0 || i>= grid.length || j<0 ||
        j >= grid[0].length || grid[i][j] != '1') {
        return;
    }
    vis[i][j] = true;
    grid[i][j] = '0'; // mark as visited
    dfs(grid, vis, i+1, j); // down
    dfs(grid, vis, i-1, j); // up
    dfs(grid, vis, i, j+1); // right
    dfs(grid, vis, i, j-1); // left
}
```

TC - O(n * m) → for — the main loop or in DFS (each cell is processed at most once either in)

SC - O(n * m) + O(n * m)
↑ recursive stack
visited +

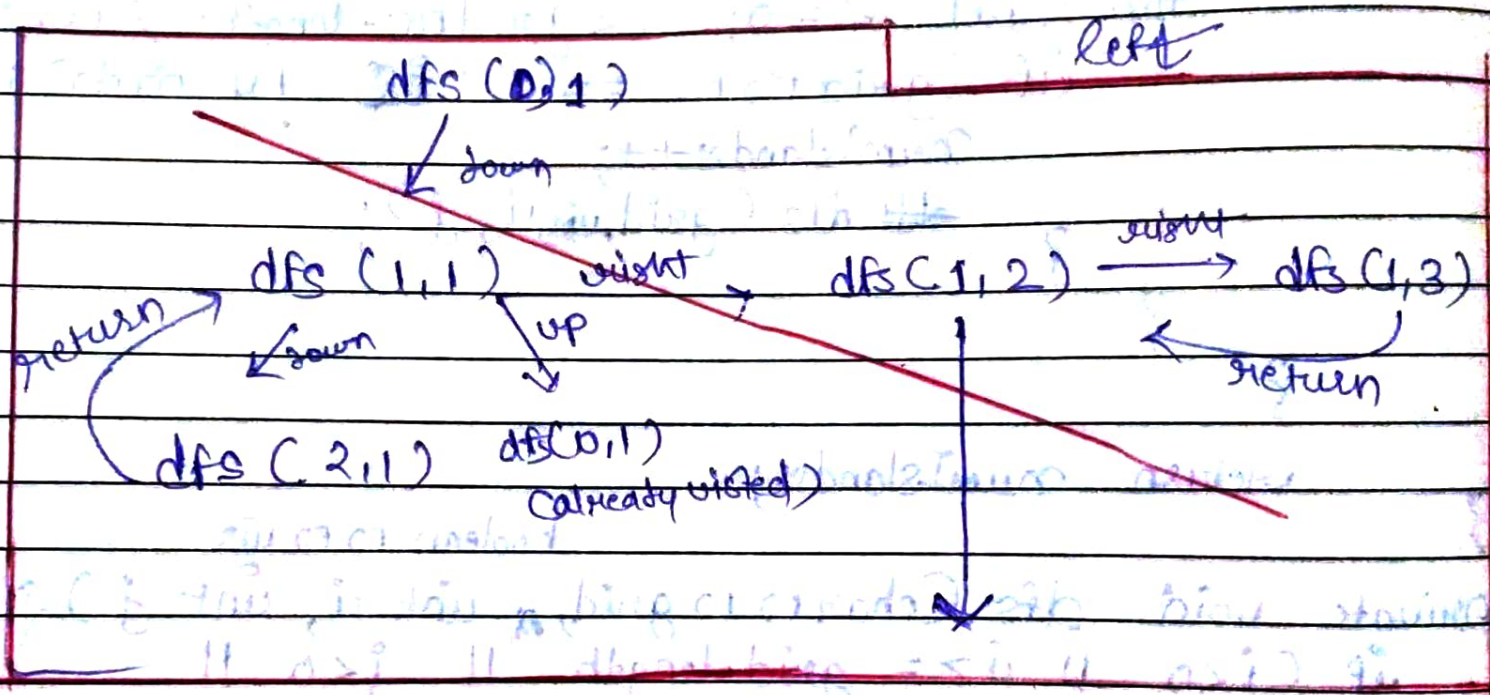
Page No.:
Date :

~~Code~~ DFS Dry Run:—

	0	1	2	3
0	0	1	1	0
1	0	1	1	0
2	0	0	1	0
3	0	0	0	0
4	1	1	0	1

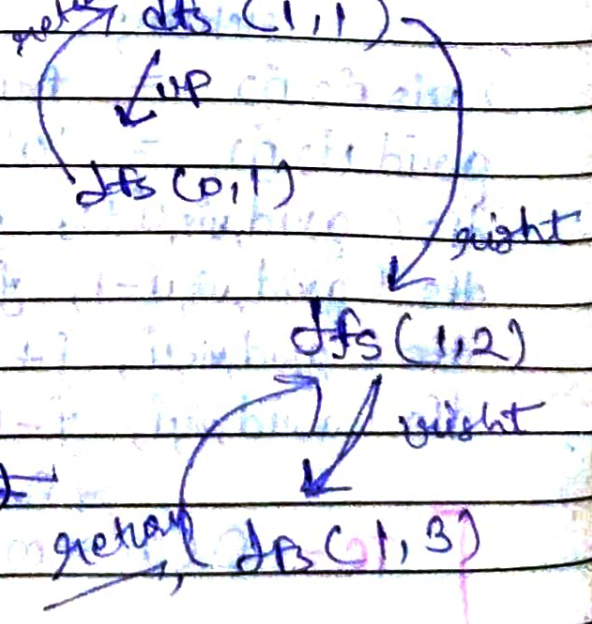
mat[0][1] == 1

down
up
right
left



dfs(0,1)
down
(dfs(1,1)) ⟹
return
down
dfs(2,1)
~~TC — O(N²)~~

~~Sc — O(N²) + O(N²)~~



Page No. :
Date :

03. Flood Fill Algorithm — direction :- +

Ex.01. sr (starting pixel row) = 1 sc (starting col pixel) = 1
newColor = 2

	0	1	2
0	1	1	1
1	1	(1)	0
2	1	0	1

{ initial color = 1 }
Color node & ⇒ its neighbours

ans ⇒

2	2	2
2	2	0
2	0	1

Ex.02. sr = 2 sc = 0 newColor = 3

	0	1	2
0	1	1	1
1	2	2	0
2	(2)	2	2

⇒ ans ⇒

1	1	1
3	3	0
3	3	3

{ initial color = 2 }

Dry :- ⇒ making replica of it

Run for visited arr

(1)

	0	1	2
0	1	1	1
1	2	2	0
2	~~2~~ 3	2	2

visited

DFS (2,0)
DFS (1,0) DFS (2,1)

(2)

	0	1	2
0	1	1	1
1	~~2~~ 3	2	0
2	3	~~2~~ 2	2

DFS (2,0)
DFS (1,0) DFS (2,1)
DFS (1,1)

Page No. :
Date :

③

	0	1	2
0	1	1	1
1	3	2	0
2	3	2	2

visited

DFS (2,0)
DFS (1,0) DFS (2,1)
DFS (1,1)
DFS (2,1)

④

	0	1	2
0	1	1	1
1	3	3	0
2	3	2	2

visited

DFS (2,0)
DFS (1,0) DFS (2,1)
DFS (1,1)
DFS (2,1)
DFS (2,2)

⑤

	1	1	1
	3	3	0
	3	3	2

visited

is it worth going? No
DFS (2,0) ✗ it is already colored
DFS (1,0) DFS (2,1)
DFS (1,1)
DFS (2,1)
DFS (2,2)

(row-1, col)

(row, col-1) ← row, col → (row, col+1)

(row+1, col)

delta change in row

drow = {-1, 0, +1, 0}

dcol = {+0, +1, +0, -1}

Page No. :
Date :

Code:-

```
private void dfs(int row, int col, int ans[][], int image[][],
    int newColor, int delRow[], int delCol[], int iniColor){
    ans[row][col] = newColor;
    int n = image.length;
    int m = image[0].length;

    for(int i=0; i<4; i++){
        int nrow = row + delRow[i];
        int ncol = col + delCol[i];
        if(nrow>=0 && nrow<n && ncol>=0 &&
            ncol<m && image[nrow][ncol] == iniColor
            && ans[nrow][ncol] != newColor){
            dfs(nrow, ncol, ans, image, newColor, delRow,
            delCol, iniColor);
        }
    }
}
```

TC - $(N*M)*4 \approx O(N*M)$

SC - $O(N*M) + O(N*M)$

```
public int[][] floodFill(int[][] image, int sr, int sc,
                         int newColor){
    int iniColor = image[sr][sc];
    int[][] ans = image;
    int delRow[] = {-1, 0, +1, 0};
    int delCol[] = {0, +1, 0, -1};

    dfs(sr, sc, image, ans, newColor, delRow, delCol,
        iniColor);
    return ans;
}
```

nrow → neighbouring row

ncol → —"— column

Page No. :

Date :

04. Rotten Oranges – direction +

Ex:

2	1	1
1	1	0
0	1	1

time = 1 ⇒

2	2	1
2	1	0
0	1	1

time = 2 sec ⇓

2	2	2
2	2	0
0	1	1

time = 3 sec ⇓

2	2	2
2	2	0
0	2	1

time = 4 sec ⇓

2	2	2
2	2	0
0	2	2

0: empty cell 1: fresh 2: rotten

ans ⇒ 4 sec

TC – O(N*m) + O(N*m*4)

SC – O(N*m)

(row-1, col) ↑

(row, col-1) ← (row, col) → (row, col+1)

↓ (row+1, col)

delRow = {-1, 0, +1, 0}

delCol = {0, +1, 0, -1}

level wise and want min.

Page No. :

Date :

Code:- //BFS → why?

```
public int orangesRotting(int grid[][]){
    int n = grid.length;
    int m = grid[0].length;

    Queue<int[]> q = new LinkedList<>();
    int vis[][] = new int[n][m];

    //Step-01: Push all initially rotten oranges to queue
    for(int i=0; i<n; i++){
        for(int j=0; j<m; j++){
            if(grid[i][j] == 2){
                q.add(new int[]{i, j, 0});
                vis[i][j] = 2;
            }
            else{
                vis[i][j] = 0;
            }
        }
    }

    int time = 0;
    int drow[] = {-1, 0, +1, 0};
    int dcol[] = {0, +1, 0, -1};

    //Step-02:- BFS traversal
    while(!q.isEmpty()){
        int front[] = q.poll();
        int r = front[0];
        int c = front[1];
        int t = front[2];
```

Page No. :
Date :

```
        time = Math.max(time, t);

        for (int i = 0; i < 4; i++) {
            int nrow = r + drow[i];
            int ncol = c + dcol[i];

            if (nrow >= 0 && nrow < n &&
                ncol >= 0 && ncol < m &&
                vis[nrow][ncol] == 0 &&
                grid[nrow][ncol] == 1) {
                q.add(new int[] {nrow, ncol, t+1});
                vis[nrow][ncol] = 2;
            }
        }
    }

    // Step-03 :- Check if any fresh orange is left
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (grid[i][j] == 1 && vis[i][j] != 2) {
                return -1;
            }
        }
    }
    return time;
}
```

TC - $O(n*m) + O(n*m*4)$

SC - $O(2*n*m)$

Page No. :
Date :

Dry Run :-

	0	1	2
0	0	1	2
1	0	1	1
2	2	1	1

(Given)

0: empty cell
1: fresh orange
2: rotten orange

Dreplica

(1) initially

	0	1	2
0			2
1			
2	2		

visited

Queue

(0,2),0
(2,0),0

maxTime = 0

(2)

	0	1	2
0			2
1			
2	2	2	

~~(2,0) +=0~~

visited

Queue

(2,1),1
(0,2),0
~~(2,0),0~~

maxTime = 0

(3)

	0	1	2
0		2	2
1			2
2	2	2	

~~(0,2) +0~~

visited

Queue

(0,1,1)
(1,2,1)
(2,1),1
~~(0,2),0~~

maxTime = 0

	0	1	2
0		2	2
1		2	2
2	2	2	2

~~(2,1),1~~

visited

Queue

(2,2,2)
~~(1,2,2)~~
(0,1,1)
(1,2,1)
~~(2,1),1~~

maxTime = 1

	0	1	2
0		2	2
1		2	2
2	2	2	2

~~(1,2) +1~~

visited (0,1,1) +1

~~(1,1,2)~~

~~(2,2,2)~~

Queue

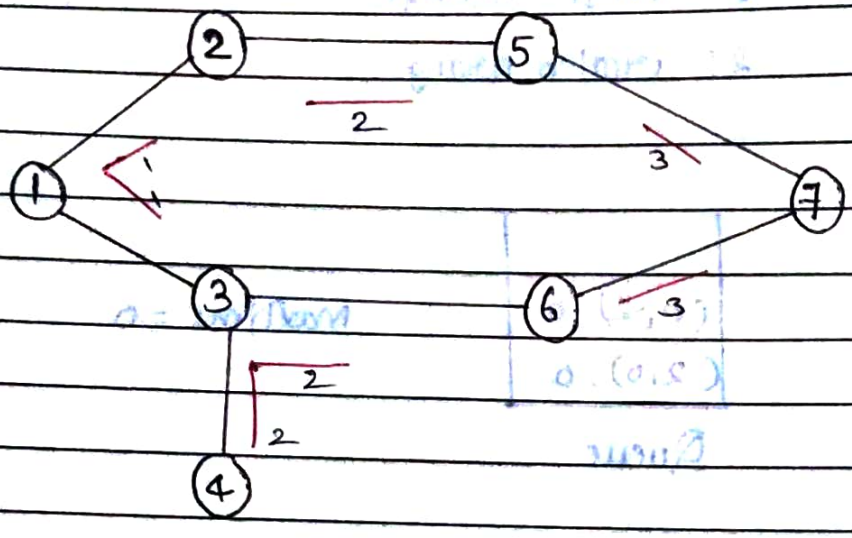
(2,2,2)
~~(2,1,2)~~
~~(0,1,1)~~
~~(1,2,1)~~

Ans.

maxTime = 2

Page No. :
Date :

05. Detect a Cycle in an Undirected Graph using BFS -



adj list -

1 →	{2,3}
2 →	{1,5}
3 →	{1,4,6}
4 →	{3}
5 →	{2,7}
6 →	{3,7}
7 →	{5,6}

* If you have started in two different paths and now you're colliding in some portion. Dude you got the cycle!

* It can have connected

Page No. :

Date :

Dry Run :-

(1)

{1, 1}

vis[] → | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 |
1 2 3 4 5 6 7

Queue → coming from nowhere

(2)

{3, 1}
{2, 1}

vis[] → | 1 | 1 | 1 | 0 | 0 | 0 | 0 | — |
1 2 3 4 5 6 7

(3)

{5, 2}
{3, 1}

vis[] → | 1 | 1 | 1 | 0 | 1 | 0 | 0 | — |
1 2 3 4 5 6 7

(4)

{6, 3}
{4, 3}
{5, 2}

vis[] → | 1 | 1 | 1 | 0 | 1 | 1 | 0 | — |
1 2 3 4 5 6 7

(5)

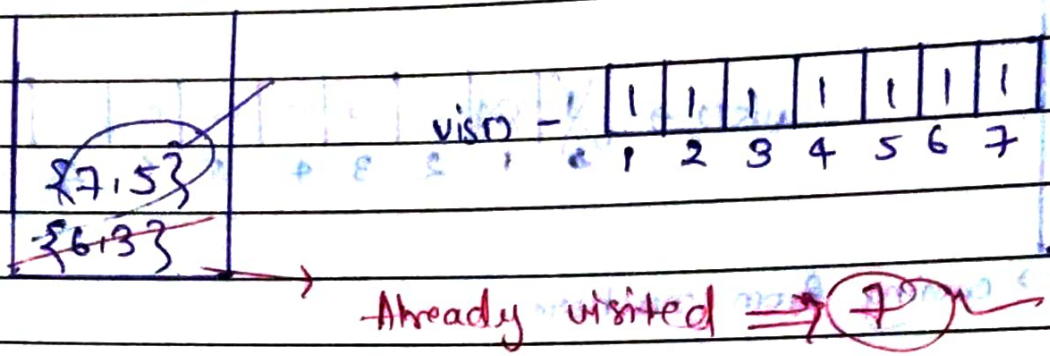
{7, 5}
{6, 3}
{4, 3}

vis[] → | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
1 2 3 4 5 6 7

Page No. :

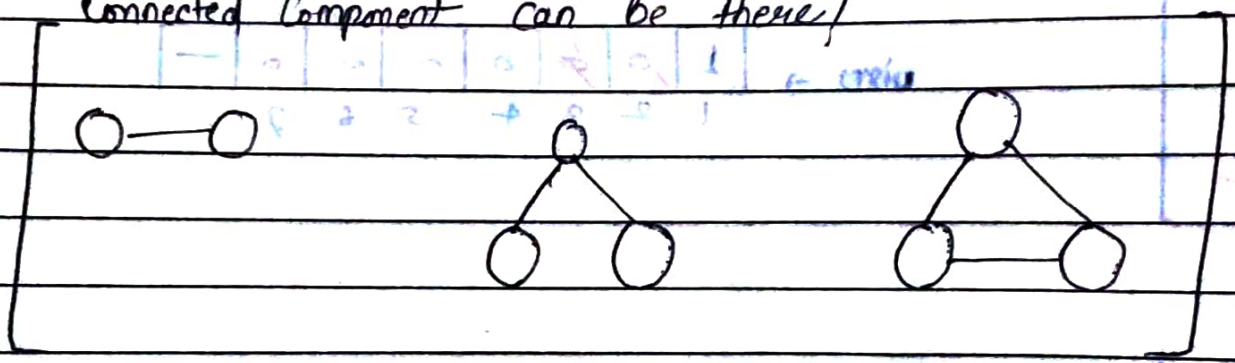
Date :

(6)



Already visited ⇒ 7

Connected Component can be there!



Page No. :
Date :

Code:-

```
public boolean checkforCycle(int src, int V,
    ArrayList<ArrayList<Integer>> adj, boolean[] vis) {

    vis[src] = true;
    Queue<Pair> q = new LinkedList<>();
    q.add(new Pair(src, -1));
    while(!q.isEmpty()){
        int node = q.peek().first;
        int parent = q.peek().second;
        q.remove();

        for(int adjacentNode : adj.get(node)){
            if(vis[adjacentNode] == false){
                vis[adjacentNode] = true;
                q.add(new Pair(adjacentNode, node));
            }
            else if(parent != adjacentNode){
                return true;
            }
        }
    }
    return false;
}
```

अगर मैं यहीं से आ रही हूँ and parent is not equal to adjacentNode.

अगर parent है और already visited है तो कोई problem नहीं है

लेकिन अगर visited है और वो parent भी नहीं है इसका मतलब किसी और ने पहुंच लिया होगा and it's graph. ✓

Page No. :
Date :

```
public boolean isCycle(int V,
        ArrayList<ArrayList<Integer>> adj) {
    boolean vis[] = new boolean[V];

    for(int i=0; i<V; i++) {
        vis[i] = false;
    }

    for(int i = 0; i<V; i++ ) {
        if(vis[i] == false) {
            if(checkForCycle(i, V, adj, vis)) {
                return true;
            }
        }
    }

    return false;
}
```

if it is 0 based indexing

connected component

FOR CONNECTED COMPONENT

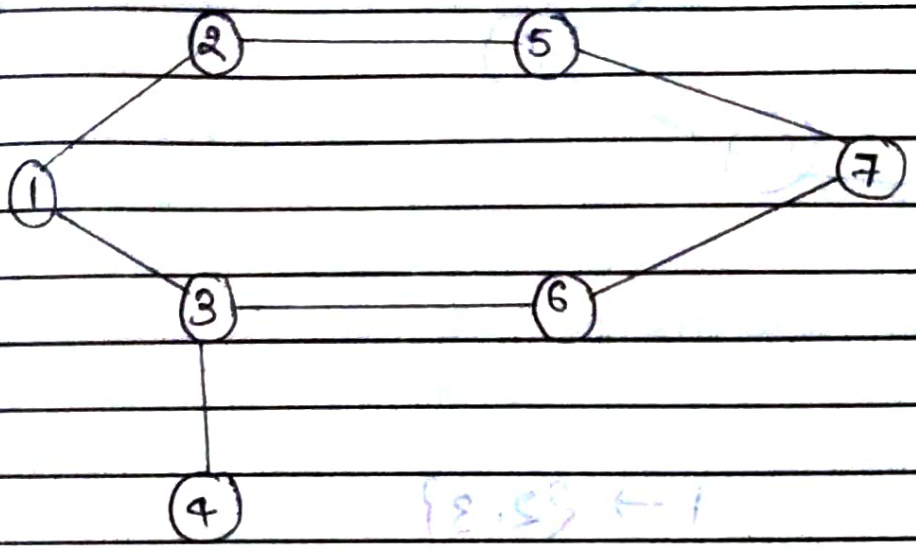
component thing.

TC - $O(N + 2E) + O(N)$

SC - $O(N) + O(N) \approx O(N)$

Page No. :
Date :

06. Detect a Cycle in an Undirected Graph using DFS :-



Adj list
1 → {2,3}
2 → {1,5}
3 → {4,1,6}
4 → {3}
5 → {2,7}
6 → {3,7}
7 → {5,6}

node parent

T → DFS (1, -1)

T → DFS (2,1)

T → DFS (5,2)

T → DFS (7,5)

True → DFS (6,7)

False ← True DFS (3,6)

DFS (4,3)

DFS (1,3)

if I got a true, no need to do further DFS calls.

DFS (3,1)

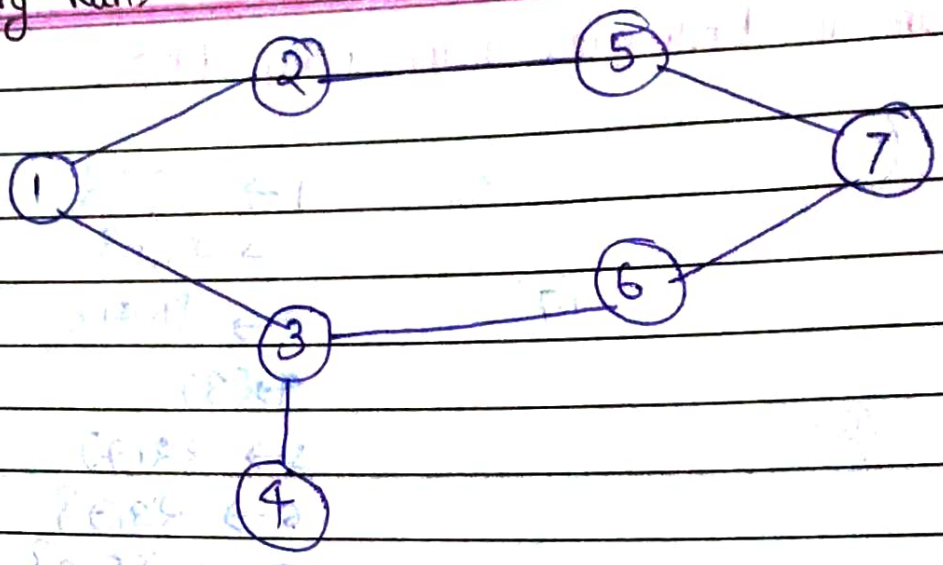
visit =

1	2	3	4	5	6	7
1	1	1	0	1	1	1

Page No. .....

Date :

Dry Run:-



dfs (1) ⇒ 1 → {2, 3}

dfs (2) ⇒ 2 → {1, 5}

T	T			T
1	2	3	4	5

dfs (5) ⇒ 5 → {2, 7}

T	T			T
1	2	3	4	5

dfs (7) ⇒ 7 → {5, 6}

T	T			T	T	T
1	2	3	4	5	6	7

dfs (6) ⇒ 6 → {3, 7}

T	T	T		T	T	T
1	2	3	4	5	6	7

why?

Page No. :
Date :

Code:-

```
private boolean dfs(int node, int parent, int vis[],
    ArrayList<ArrayList<Integer>> adj) {

    vis[node] = 1;
    for (int adjacentNode : adj.get(node)) {
        if (vis[adjacentNode] == 0) {
            if (dfs(adjacentNode, node, vis, adj) == true) {
                return true;
            }
        }
        else if (adjacentNode != parent) {
            return true;
        }
    }
    return false;
}
```

SC- O(N) + O(N) ≈ O(N)
TC- O(N+2E) + O(N)

```
// Function to detect cycle in an undirected graph.
public Boolean isCycle(int V, ArrayList<ArrayList<Integer>> adj) {
    int vis[] = new int[V];

    for (int i = 0; i < V; i++) {          // loop for connected component...
        if (vis[i] == 0) {
            if (dfs(i, -1, vis, adj) == true) {
                return true;
            }
        }
    }
    return false;
}
```

G-13

Page No. :
Date :

07. Distance of nearest cell having 1 — 0+1 matrix :-

1	0	1			0	1	0
1	1	0	⇒	ans.	0	0	1
1	0	0			0	1	2

direction (←↑→↓)

BFS (why?) It will work step by step

Dry Run :-

(1) Initially :-

	0	1	2
0	0	0	0
1	0	①	0
2	①	0	①

(given)

0	0	0	0
1	0	0	0
2	0	0	0

visited

distance

(2,2),0
(2,0),0
(1,1),0

Queue ..

(2) (1,1),0 → (1,0),1

(2,1),1
(1,2),1
(0,1),1
(2,2),0
(2,0),0

	0	1	2
0			
1		0	
2			

distance

0	0	1	0
1	1	0	1
1	0	1	1

visited

Page No. :

Date :

③ (2,0), 0

0	1	0
1	1	1
1	1	1

visited

(i) already vis
(ii) — " —
(iii) No
(iv) No

(1,0), 1
(2,1), 1
(1,2), 1
(0,1), 1
(2,2), 0

Queue

	0	1	2
0			
1		0	
2	0		

distance

0	0	0
0	1	0
1	0	1

given

S : (0,0)
S : (S,0)
1 : (0,1)
1 : (1,2)

④ (2,2), 0

0	0	0
0	1	0
1	0	1

given

(i) already visited
(ii) —X—
(iii) No
(iv) already visited

(1,0), 1
(2,1), 1
(1,2), 1
(0,1), 1

Queue

0			
1		0	
2	0		0

distance

⑤ (0,1), 1

0	0	0
0	1	0
1	0	1

(i) No → (0,2), 2
(ii) ~~already visited~~
(iii) — " —
(iv) (0,0), 2

(0,0), 2
(0,2), 2
(1,0), 1
(2,1), 1
(1,2), 1

Queue

0		1	
1		0	
2	0		0

dist

Page No. :
Date :

(6) (1,2), 1

(i) already visited
(ii) No
(iii) already visited
(iv) —"— —"—

	0	1	2
0		1	
1		0	1
2	0		0

(distance).

	0	1	2
0	0	0	0
1	0	1	0
2	1	0	1

(given).

Queue
(0,0), 2
(0,2), 2
(1,0), 1
(2,1), 1

(7) (2,1), 1

	0	1	2
0		1	
1		0	1
2	0	1	0

(distance).

	0	1	2
0	0	0	0
1	0	1	0
2	1	0	1

(given).

(i) already visited
(ii) —"— —"—
(iii) ✗ No
(iv) already visited.

(8) (1,0), 1

	0	1	2
0		1	
1	1	0	1
2	0	1	0

(distance).

	0	1	2
0	0	0	0
1	0	1	0
2	1	0	1

(given).

(i) already visited
(ii) —"— —"—
(iii) —"— —"—
(iv) No ✗

Page No. :
Date :

(9) (0,2), 2

	0	1	2
0		1	2
1	1	0	1
2	0	1	0

(distance)

0	0	0
0	1	0
1	0	1

(given)

(i) Invalid / No (∞)
(ii) X
(iii) already visited
(iv) ~~X~~

(10) (0,0), 2

	0	1	2
0	2	1	2
1	1	0	1
2	0	1	0

(distance) Ans.

0	0	0
0	1	0
1	0	1

(given)

(i) No X
(ii) Already visited
(iii) —"—
(iv) No X.

The Queue has become Empty. (You have visited everything)

row-1, col

row, col-1 ← (row, col) → row, col+1

row+1, col

{-1, 0, +1, 0}

{0, +1, 0, -1}

LeetCode ke Code

Page No. :

Date :

Code :-

```
public int updateMatrix[][] (int mat[][]){
    int n = mat.length;
    int m = mat[0].length;

    int[][] dist = new int[n][m];
    boolean[][] vis = new boolean[n][m];

    Queue<int[]> q = new LinkedList<>();

    for (int i=0; i<n; i++){
        for (int j=0; j<m; j++){
            if (mat[i][j] == 1){
                dist[i][j] = Integer.MAX_VALUE;
            } q.offer(new int[]{i, j, 0});
            else {vis[i][j] = true;
                q.offer(new int[]{i, j});
                vis[i][j] = true;
            } vis[i][j] = false;
        }
    }

    int dr[] = {-1, 0, 1, 0};
    int dc[] = {0, 1, 0, -1};
```

Page No. :
Date :

```
while (!q.isEmpty()) {
    int curr[] = q.poll();
    int row = curr[0];
    int col = curr[1];
    int steps = curr[2];
    for (int i = 0; i < 4; i++) {
        int nrow = row + dr[i];
        int ncol = col + dc[i];

        if(nrow >= 0 && nrow < n &&
           ncol >= 0 && ncol < m &&
           !vis[nrow][ncol]) {
            vis[nrow][ncol] = true;
            q.offer(new int[]{nrow, ncol, steps+1});
        }
    }
}
return dist;
}
```

① q.remove();
② dist[row][col] = steps;

TC - O(N*M*4) + O(N*M)
SC - O(N*M) + O(N*M)

Page No. :
Date :

08. Surrounded Regions / Replace O's with X's

direction (↕)

(we observed) :- Set of X connected to a boundary 'O' cannot be converted...

DFS (why??) :- we want to go in depth (in all the connected connection

Ex:

X X X X
X O X X
X O O X
X O X X
X X O O

- Start from boundary O's & mark them that they will not be converted & convert the remain O's.

⇒ Dry Run:-

01. Initial step:-

X X X X X
X O O X O
X X O X O
X O X O X
O O X X X
(given matrix)

O O O O O
O O O O O
O O O O O
O O O O O
O O O O O
visited

Page No. :
Date :

2. In first row :-

```
X  X  X  X  X   → No 0's available in 1st row
X  0  0  X  0
X  X  0  X  0
X  0  X  0  X
0  0  X  X  X
```

Given

3. In first column :-

```
   0  1  2  3  4
0  X  X  X  X  X
1  X  0  0  X  0
2  X  X  0  X  0
3  X  0  X  0  X
4  0  0  X  X  X
```

Given

DFS (4,0)
↙
DFS (4,1)
↙
DFS (3,1)

```
0  0  0  0  0
0  0  0  0  0
0  0  0  0  0
0  1  0  0  0
1  1  0  0  0
```

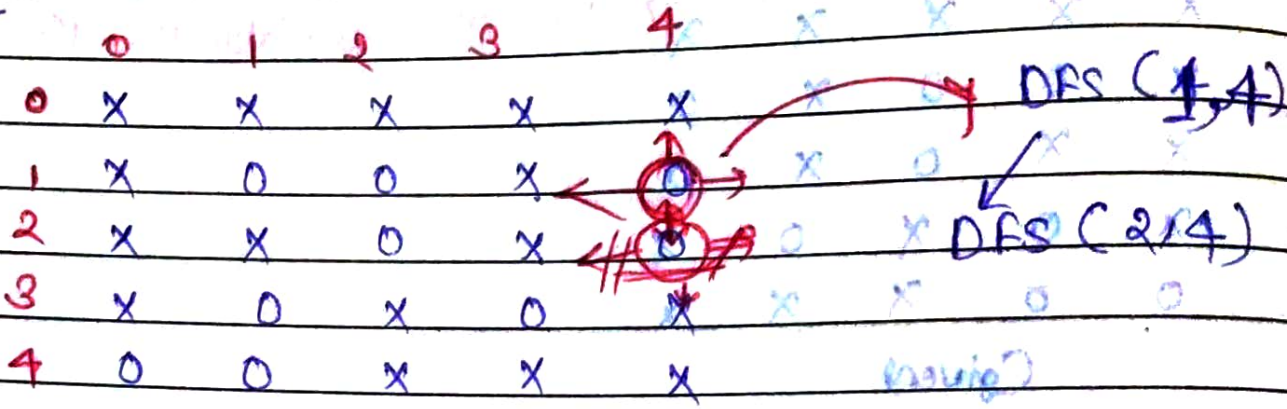
visited.

row-1, col
↑
row, col → row, col+1
↓
row+1, col

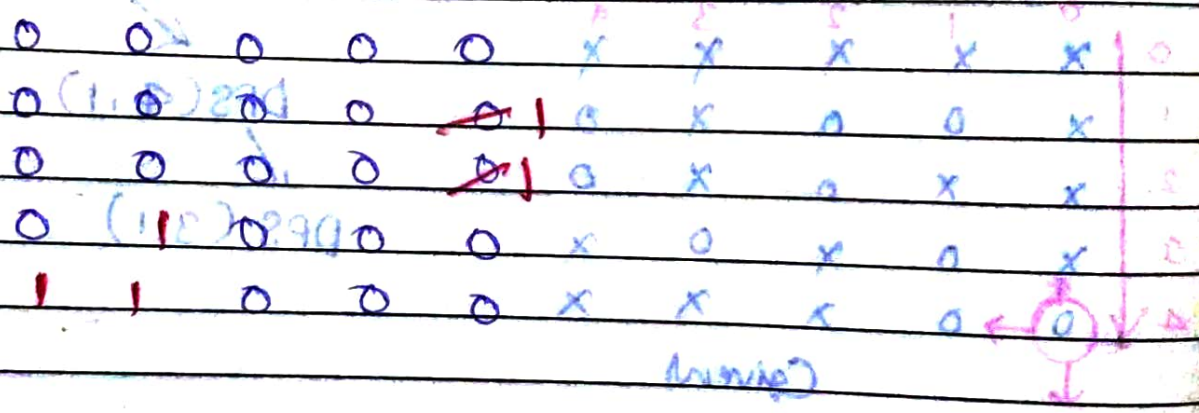
Page No. :
Date :

In last Row ⇒ Already visited...

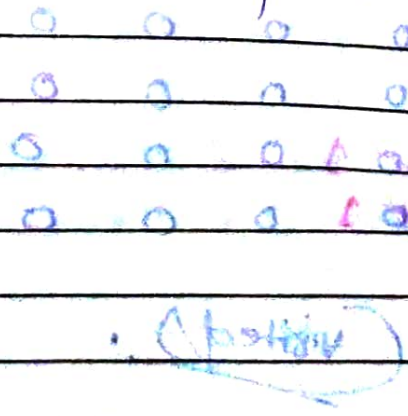
04. Right side Col (last Col)



Visited



01- From Boundary find 0's & traverse & mark all 0's connected to it.



Code :-

Page No. :
Date :

```
public void solve(char[][] board){
    int n = board.length;
    int m = board[0].length;
    int vis[][] = new int[n][m];
    int delrow[] = {-1, 0, 1, 0};
    int delcol[] = {0, 1, 0, -1};

    // mark boundary connected 'O's
    for(int j = 0; j<m; j++){ // O(m)
1st Row  if(board[0][j] == 'O' && vis[0][j] == 0){
        dfs(0, j, vis, board, delrow, delcol);
    }
last Row if(board[n-1][j] == 'O' && vis[n-1][j] == 0){
        dfs(n-1, j, vis, board, delrow, delcol);
    }
}

for(int i = 0; i<n; i++){ // O(n)
1st Col  if(board[i][0] == 'O' && vis[i][0] == 0){
        dfs(i, 0, vis, board, delrow, delcol);
    }
last Col if(board[i][m-1] == 'O' && vis[i][m-1] == 0){
        dfs(i, m-1, vis, board, delrow, delcol);
    }
}

for(int i = 0; i<n; i++){
    for(int j = 0; j<m; j++){
        if(board[i][j] == 'O' && vis[i][j] == 0){
            board[i][j] = 'X';
}}}
```

Page No. :
Date :

```
private void dfs (int row, int col, int [][] vis,
    char [][] board, int delrow [], int delcol []) {
    vis [row] [col] = 1;
    int n = board.length;
    int m = board[0].length;

    for (int i = 0; i<4; i++) {
        int nrow = row + delrow [i];
        int ncol = col + delcol [i];
        if (nrow >= 0 && nrow < n &&
            ncol >= 0 && ncol < m &&
            board [nrow] [ncol] == 'O' &&
            vis [nrow] [ncol] == 0) {

            dfs (nrow, ncol, vis, board, delrow,
                delcol);
        }
    }
}
```

TC — O(N*M) + O(N) + O(N*M)*4

SC — O(N*M) + O(N*M)

Auxiliary ...

MULTI-SOURCE BFS

Page No. :
Date :

09. Number of Enclaves - direction (4)
Count the no. of lands (1).

0 0 0 1 1
0 1 1 0
0 1 1 0 ⇒ Ans. (4) ∵ they will not go out of the boundary
0 0 0 1
0 1 1 0

0 0 0 1
0 1 1 0
0 1 1 0 → Ans. 0
0 0 0 1 All will go out of the boundary
0 1 1 0

(we observed) :- Boundary 1's & then connected all the 1's connected to it.

(+,+)
(-,+)
(+,-)

{row-1, col}
{row, col-1} ← {row, col} → {row, col+1}
{row+1, col}

(-,+)
(+,+)
(-,-)

I can also solve this using DFS As I did in previous question.

Page No. :

Date :

Dry Run :-

put down all the boundary guys

①

	0	1	2	3	4
0	0	0	0	1	1
1	0	0	1	1	0
2	0	1	0	0	0
3	0	1	1	0	0
4	0	0	0	1	1

(given)

				1	1
					0
					0
					0
				1	1

(visited)

Queue
4,4
4,3
0,4
0,3

multisource BFS as we start with more than 1 start points.

② (0,3)

0	0	0	1	1	
0	0	1	0	0	
0	1	0	0	0	
0	1	1	0	0	
0	0	0	1	1	

(given)

				1	1
					0
					0
					0
				1	1

(visited)

Queue
(0,3)
(4,4)
(4,3)
(0,4)

③ (0,4)

0	0	0	1	1	
0	0	1	1	0	
0	1	0	0	0	
0	1	1	0	0	
0	0	0	1	1	

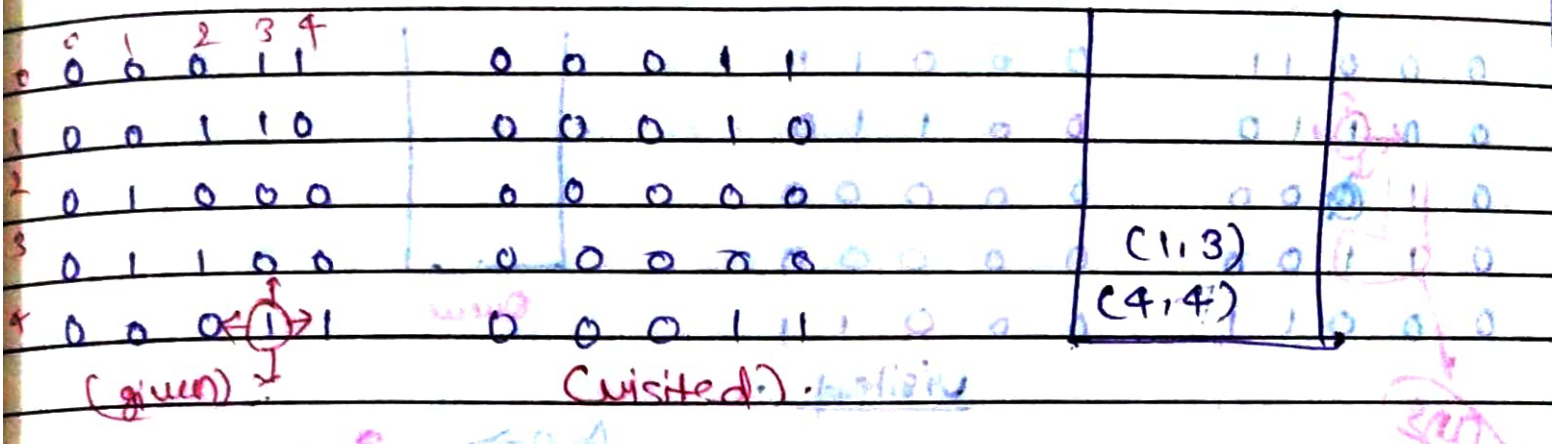
				1	1
				1	0
					0
					0
				1	1

Queue
(1,3)
(4,4)
(4,3)

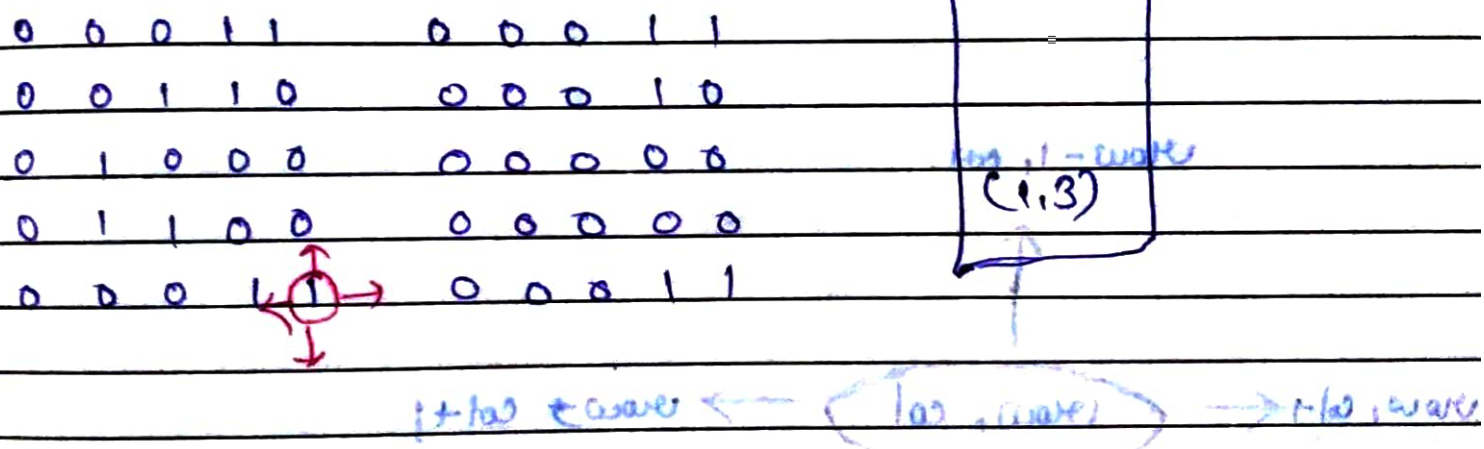
Page No. :

Date :

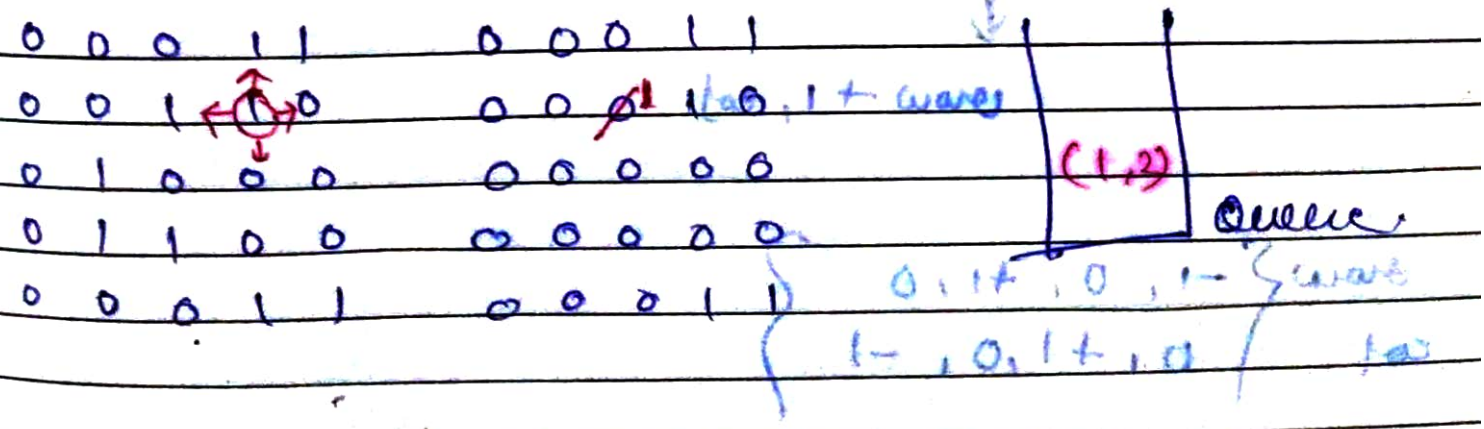
④ (4,3)



⑤ (4,4)



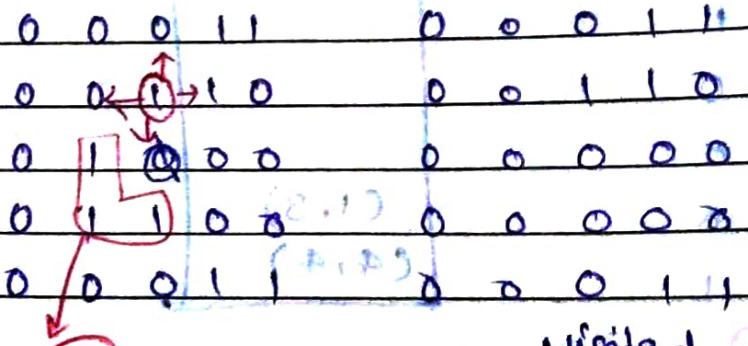
⑥ (1,3)



Page No. :

Date :

(7) (1, 2)



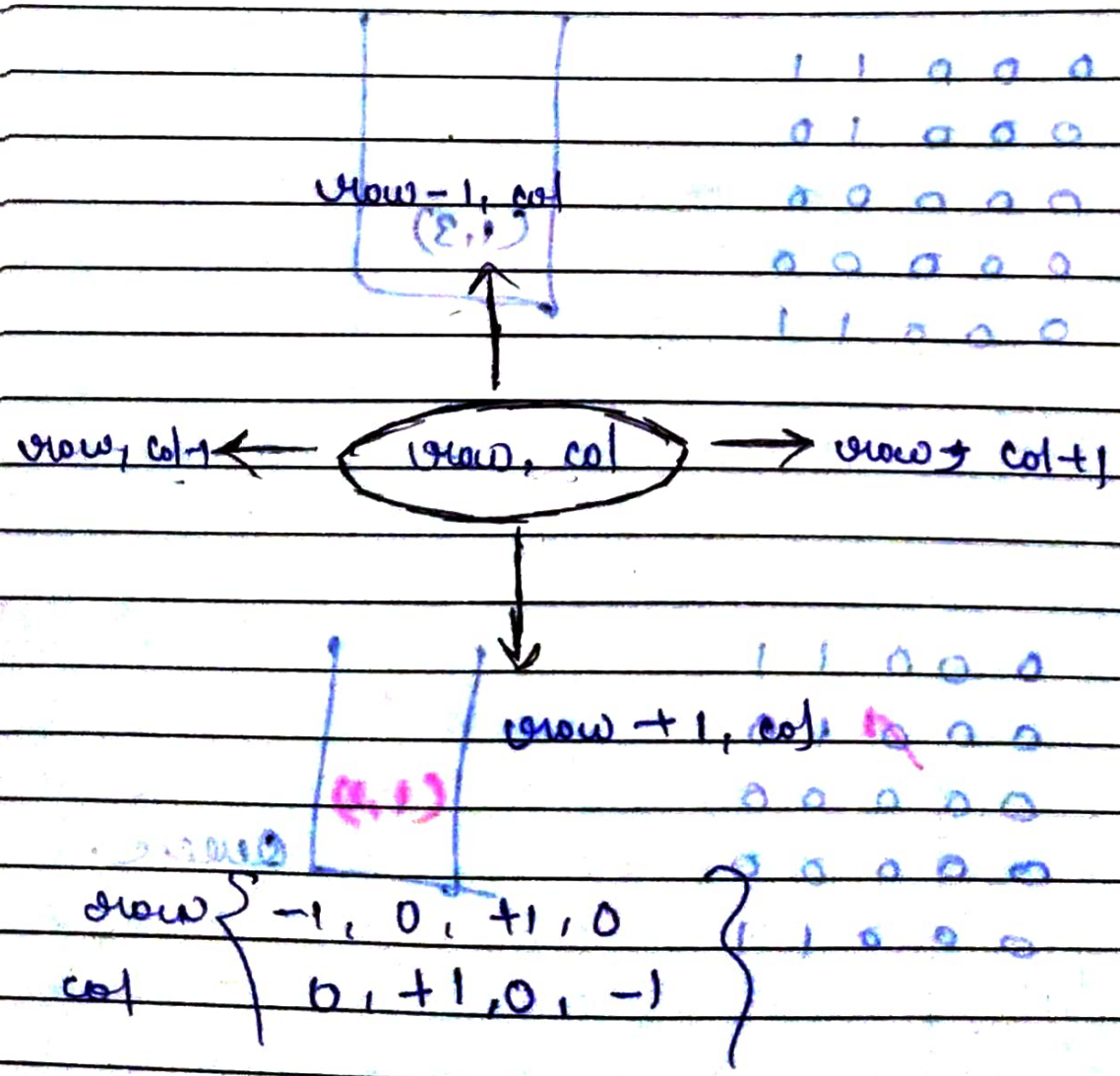
Queue.

Visited.

Ans

Now Queue is Empty

Ans ⇒ 3.



Code: —

Page No. :
Date :

```
public int numberOfEnclaves(int grid[][]) {
    Queue<int[]> q = new LinkedList<>();
    int n = grid.length;
    int m = grid[0].length;

    int vis[][] = new int[n][m];

    for (int i = 0; i<n; i++) {
        for (int j = 0; j<m; j++) {
            // first row, first col, last row, last col
            if (i == 0 || j == 0 || i == n-1 ||
                j == m-1) {
                if (grid[i][j] == 1) {
                    q.offer(new int[] {i, j});
                    vis[i][j] = 1;
                }
            }
        }
    }

    int delRow[] = {-1, 0, +1, 0};
    int delCol[] = {0, +1, 0, -1};

    while (q.isEmpty()) {
        int row = q.poll();
        int curr[] = q.poll();
        int row = curr[0];
        int col = curr[1];
                                    → q.removed
```

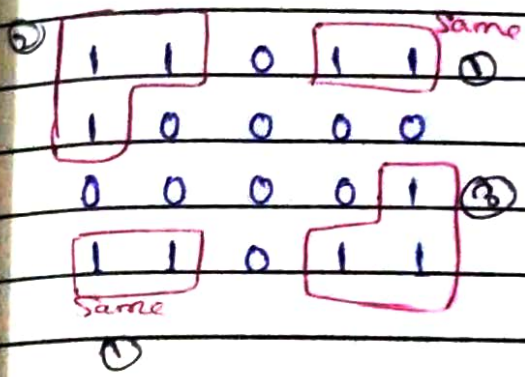
```
// traverses all 4 directions
for (int i = 0; i < 4; i++) {
    int nrow = row + delRow[i];
    int ncol = col + delCol[i];

    if (nrow >= 0 && nrow < n &&
        ncol >= 0 && ncol < m &&
        vis[nrow][ncol] == 0 &&
        grid[nrow][ncol] == 1) {
        q.offer(new int[]{nrow, ncol});
        vis[nrow][ncol] = 1;
    }
}
int cnt = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (grid[i][j] == 1 &&
            vis[i][j] == 0) {
            cnt++;
        }
    }
}
return cnt;
}
```

TC - O(N*M) * 4 + O(N*M)

SC - O(N*M) + O(N*M)

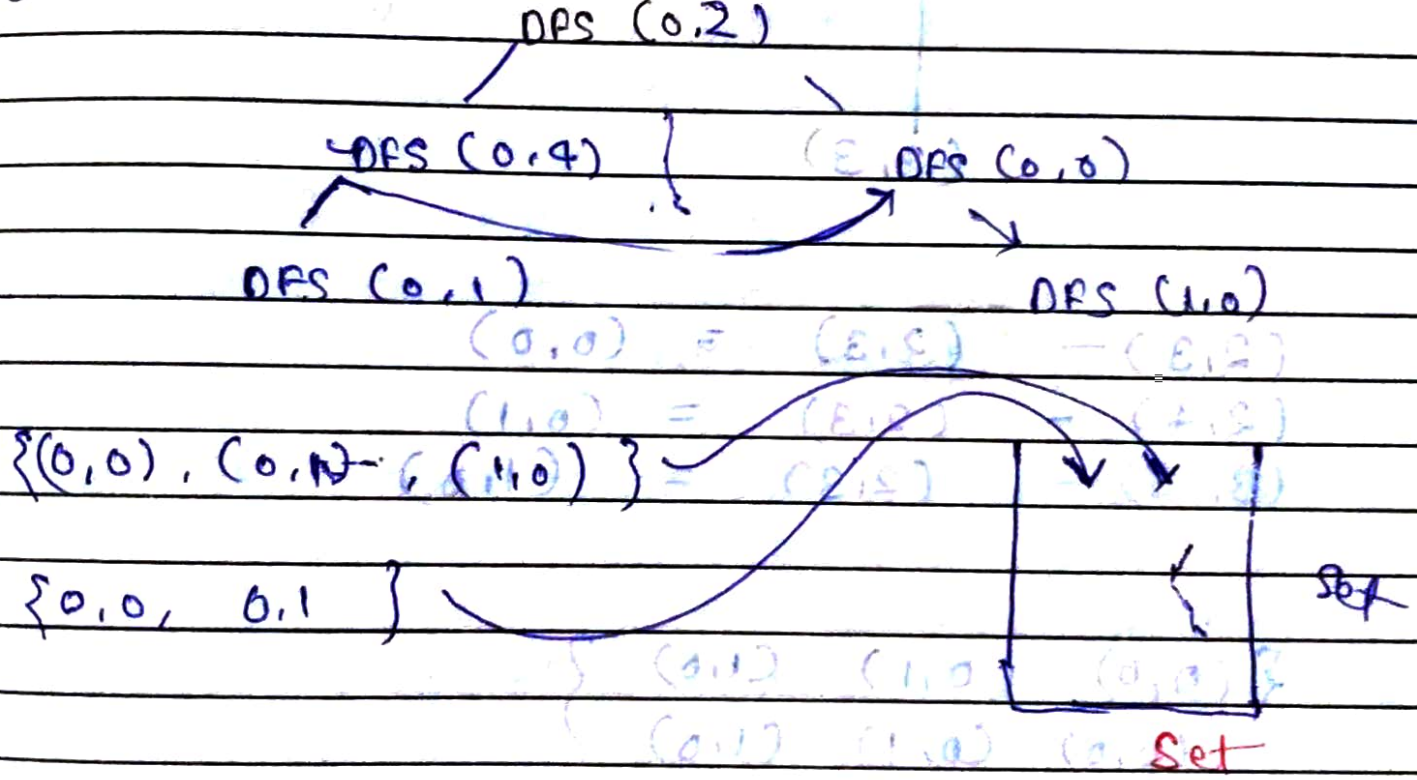
Page No. :
Date :

10. Numbers of Distinct Islands – (Premium Qs)



Ans. 3

Dry Run:



	0	1	2	3	4
0	1	1	0	1	1
1	1	0	0	0	0
2	0	0	0	1	1
3	1	1	0	1	0

Page No. :

Date :

(Base) (0,0) —— (0,1)

|

(1,0)

(Base) (2,3) —— (2,4)

|

(3,3)

(2,3) − (2,3) = (0,0)

(2,4) − (2,3) = (0,1)

(3,3) − (2,3) = (1,0)

{(0,0) (0,1) (1,0)}

(0,0) (0,1) (1,0)

int row0 = base idx
int col0 = base idx

Page No. :
Date :

Code :-

```
private void dfs (int row, int col, int [][] vis,
int [][] grid, List<String> vec, int row0, int col0) {

    vis[row][col] = 1;
    vec.add ((row - row0) + " " + (col - col0));

    int n = grid.length;
    int m = grid[0].length;

    int [] delRow = {-1, 0, +1, 0};
    int [] delCol = {0, -1, 0, +1};

    for (int i = 0; i < 4; i++) {
        int nrow = row + delRow[i];
        int ncol = col + delCol[i];

        if (nrow >= 0 && nrow < n &&
            ncol >= 0 && ncol < m &&
            vis[nrow][ncol] == 0 &&
            grid[nrow][ncol] == 1) {

            dfs (nrow, ncol, vis, grid, vec,
                 row0, col0);
        }
    }
}
```

Page No. :
Date :

```
public int countDistinctIslands(int[][] grid) {
    int n = grid.length;
    int m = grid[0].length;
    int[][] vis = new int[n][m];

    Set<List<String>> st = new HashSet<>();

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (vis[i][j] == 0 && grid[i][j] == 1) {
                List<String> vec = new ArrayList<>();
                dfs(i, j, vis, grid, vec, i, j);
                st.add(vec);
            }
        }
    }
    return st.size();
}
```

O(N*M)

TC - O(N*M) + O(N*M*4) * log(N*M)

↓ for set

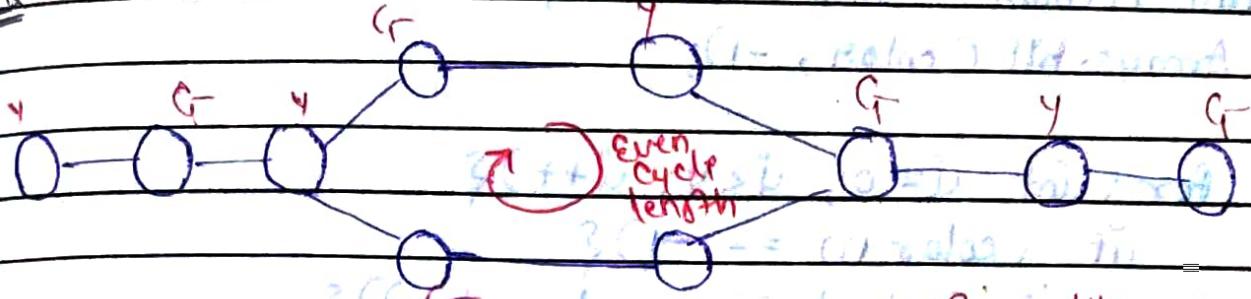
SC - O(N*M) + O(N*M)

Page No. :
Date :

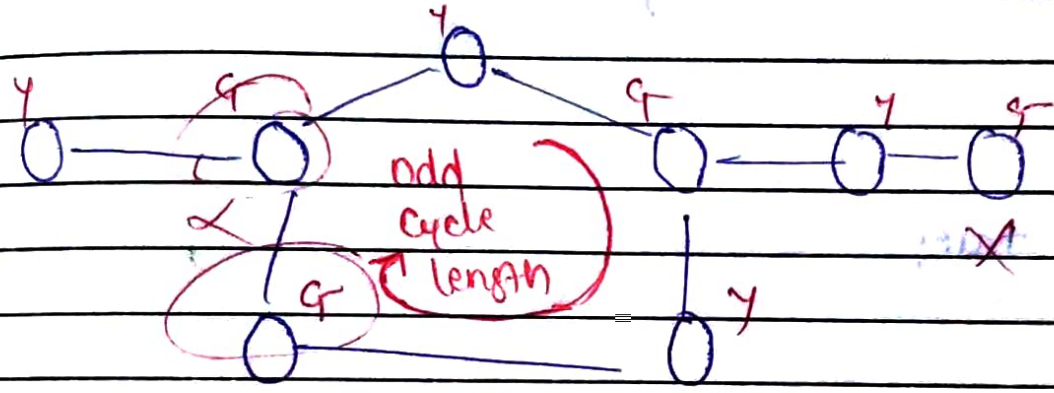
11. Bipartite Graph - (using BFS)

↳ color the graph with 2 color such that no adjacent nodes have same color.

Ex:



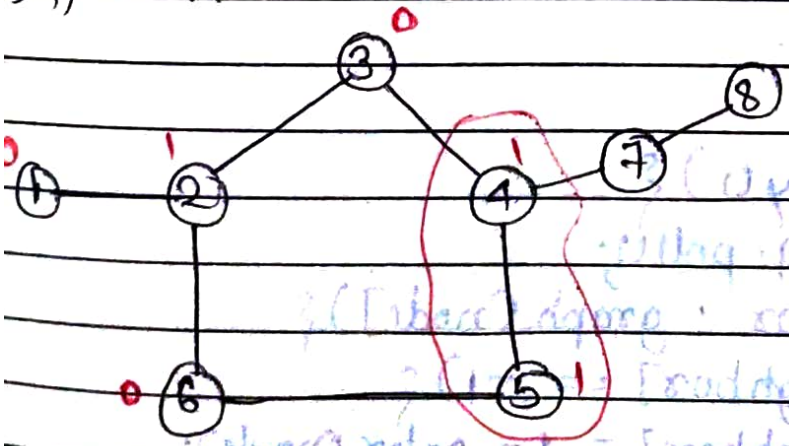
Bipartite graph.



✗ Bipartite graph

* Linear Graphs with no cycle are always Bip

DRY RUN:-



	1	2	3	4	5	6	7	8
Color -	0	1	0	-1	-1	0	-1	-1

0 & 1

node → 1
node → 2
node → 3
node → 6
node → 4 ✗

Queue
5
4
~~6~~
~~3~~
~~2~~
~~1~~

1 → {2}
2 → {1,3,6}
3 → {2,4}
4 → {3,5,7}
5 → {4,6}
6 → {2,5}
7 → {4,8}
8 → {7}

Page No. :
Date :

Code :-

```
public boolean isBipartite(int[][] graph){
    int V = graph.length;
    int[] color = new int[V];
    Arrays.fill(color, -1);

    for(int i=0; i<V; i++){
        if(color[i] == -1){
            if(!check(i, graph, color)){
                return false;
            }
        }
    }
    return true;
}

private boolean check(int start, int[][] graph,
    int[] color){
    Queue<Integer> q = new LinkedList<>();
    q.add(start);
    color[start] = 0;

    while(!q.isEmpty()){
        int node = q.poll();
        for(int neighbor : graph[node]){
            if(color[neighbor] == -1){
                color[neighbor] = 1 - color[node];
                q.add(neighbor);
            }
```

```
        else if (color[neighbor] == color[node]){
            return false;
        }
    }
}

return true;
}
```

TC- O(V+2E)

SC- O(V) + O(V) → Queue grow up to V in the worst case

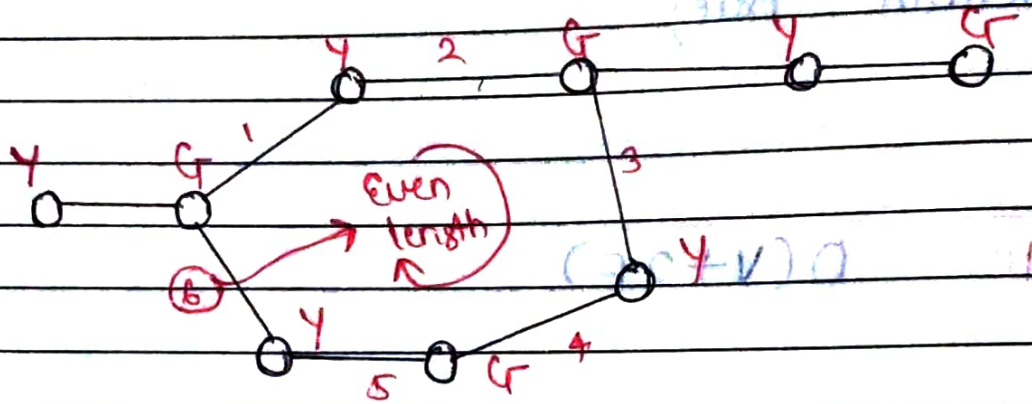
↓

V = array of size V

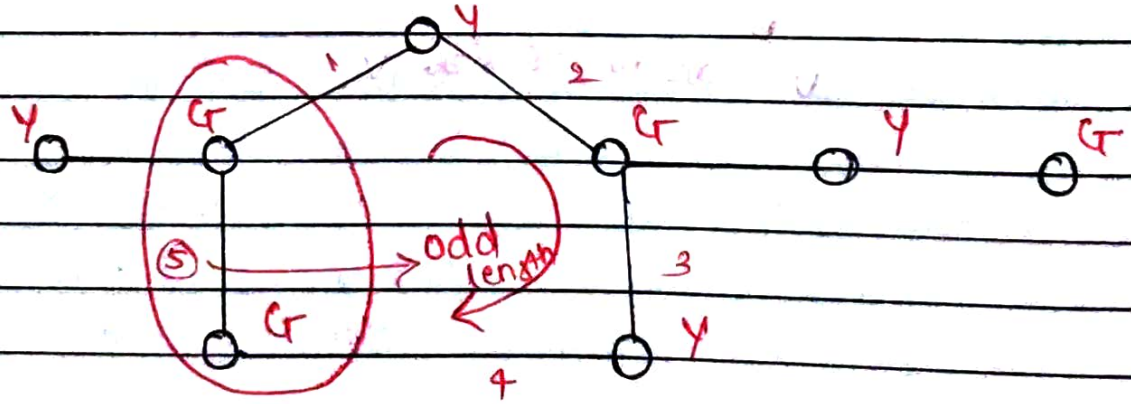
Page No. :
Date :

12. Bipartite Graph (using BFS) :- A graph that can be colored with 2 colors such that no 2 adjacent nodes have the same color.

Ex 01. Bipartite Graph (✓) yes -



Ex. 02- Bipartite Graph (X) No -



Cycle

odd length ⟲ → ✗ Bipartite

Even length ⟲ → ✓ Bipartite.

DFS Striver :-

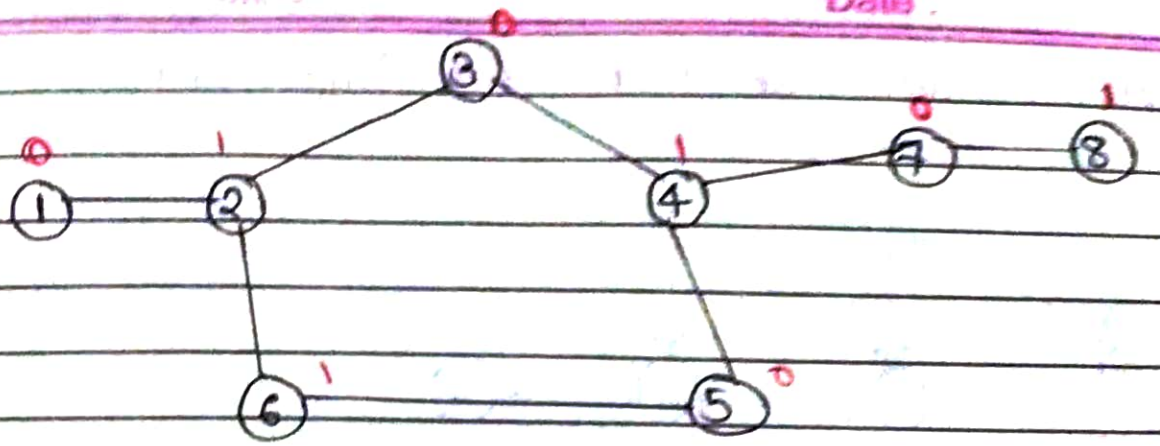
```
public boolean isBipartite(int V,
    ArrayList<ArrayList<Integer>> adj){

    int color[] = new int[V];
    Arrays.fill(color, -1);

    for(int i=0; i<V; i++){
        if(color[i] == -1){
            if(!dfs(i,0,color,adj)){
                return false;
            }
        }
    }

    return true;
}
public boolean dfs(int node, int currColor,
    int color[], ArrayList<ArrayList
    <Integer>> adj){
    color[node] = currColor;

    for(int neighbor: adj.get(node)){
        if(color[neighbor] == -1){
            if(!dfs(neighbor, 1-currColor,
                color, adj)){
                return false;
            }
        }
        else if(color[neighbor] == currColor){
            return false;
        }
    }

    return true;
}
```

TC - O(V+2E)
SC - O(V)

Code DFS (Bipartite Graph) -

```
public boolean isBipartite(int[][] graph){
    int n = graph.length;
    int[] colors = new int[n];

    for(int i=0; i<n; i++){
        if(colors[i]==0 &&
            !dfs(graph, colors, i, 1)){
            return false;
        }
    }
    return true;
}
private boolean dfs(int[][] graph, int[] colors,
                    int node, int color){
    if(colors[node] != 0){
        return colors[node] == color;
    }

    colors[node] = color;

    for(int neighbor : graph[node]){
        if(!dfs(graph, colors, neighbor,
                -color)){
            return false;
        }
    }
    return true;
}
```

TC - O(V+2E)
SC - O(V)



DFS (1,0) → false
DFS (2,1) → false
DFS (3,0) → false
DFS (4,1) → false
DFS (7,0)
DFS (8,1)
DFS (5,0) → false
DFS (6,1)

short = 1

1 → {2}
2 → {1,3}
3 → {2,4}
4 → {3,7,5}
5 → {4,6}
6 → {2,5}
7 → {4,8}
8 → {7}

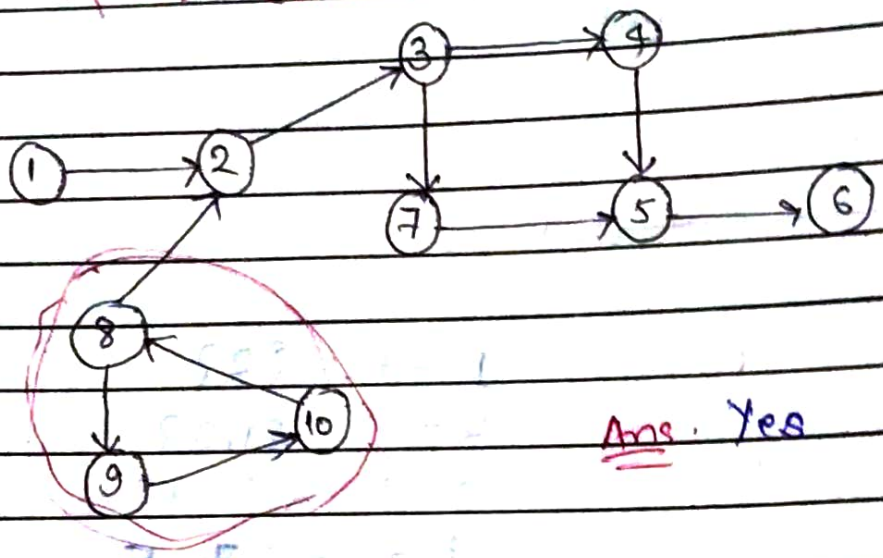
6 says d have a adjacent Node which has same color that means it is not Bipartite

Color =

0	1	2	3	4	5	6	7	8
-1	0	1	0	1	1	1	0	1

Page No. .....
Date :

13. Detect a cycle in a Directed Graph (DFS) -
{207. Course Schedule}



Ans. Yes

* On the same path, node has to be visited again.

Dis Algo using DFS technique will fail why?
because -

DFS (1)
|
DFS (2)
|
DFS (3) ——— DFS (7)
| |
DFS (4) DFS (5)
|
DFS (5)

On the same path, Node has to be visited again Then I can say it has a cycle.

This will return as a cycle but it is not due to directed edges

Page No. :
Date :

vis →

0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0

When you return / come back make pathVis to 0 again

pathVis →

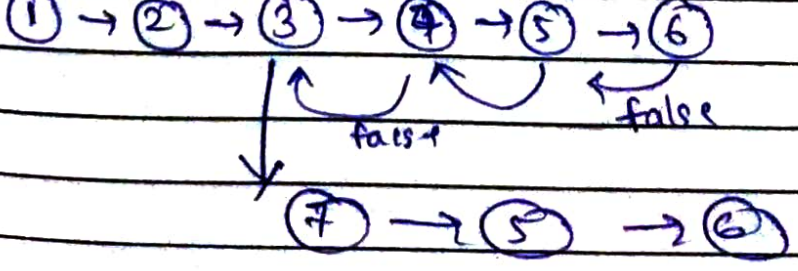
0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0

It is for the same path...

Code :- DFS :-

```
private boolean dfsCheck(int node, List<List<Integer>> adj,
        int[] vis, int[] pathVis) {
    vis[node] = 1;
    pathVis[node] = 1;

    for (int it : adj.get(node)) {
        if (vis[it] == 0) {
            if (dfsCheck(it, adj, vis, pathVis)) {
                return true;
            }
        } else if (pathVis[it] == 1) {
            return true;
        }
    }

    pathVis[node] = 0;
    return false;
}
```

visit करते रहो लेकिन आगर वापस आते है तो visited में जैसा का वैसा but pathVisited से remove या marking हटा दो।



Date :

```
public boolean canFinish(int numCourses,
        int[][] prerequisites) {
List<List<Integer>> adj = new ArrayList<>();

for (int i = 0; i < numCourses; i++) {
    adj.add(new ArrayList<>());
}

int[] vis = new int[numCourses];
int[] pathVis = new int[numCourses];

for (int i = 0; i < numCourses; i++) {
    if (vis[i] == 0) {
        if (dfsCheck(i, adj, vis, pathVis)) {
            return false;
        }
    }
}

return true;
}
```

directed graph one edge only

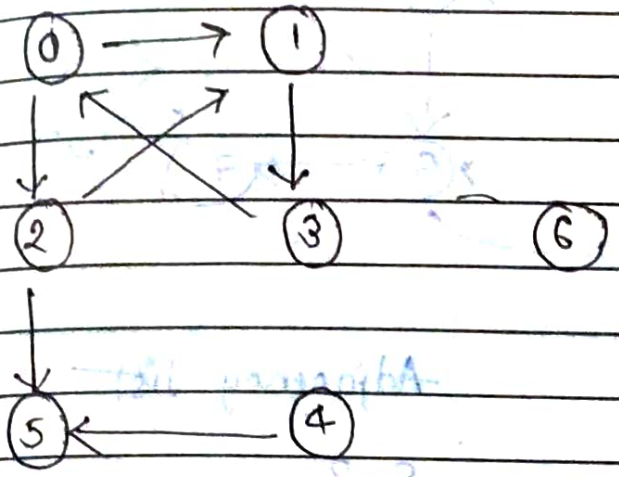
TC - O(V+E)

SC - O(2N)

Striver G-20

Page No. :
Date :

14. Find Eventual Safe States - (DFS)



Ans. [2, 4, 5, 6]
Ascending order.

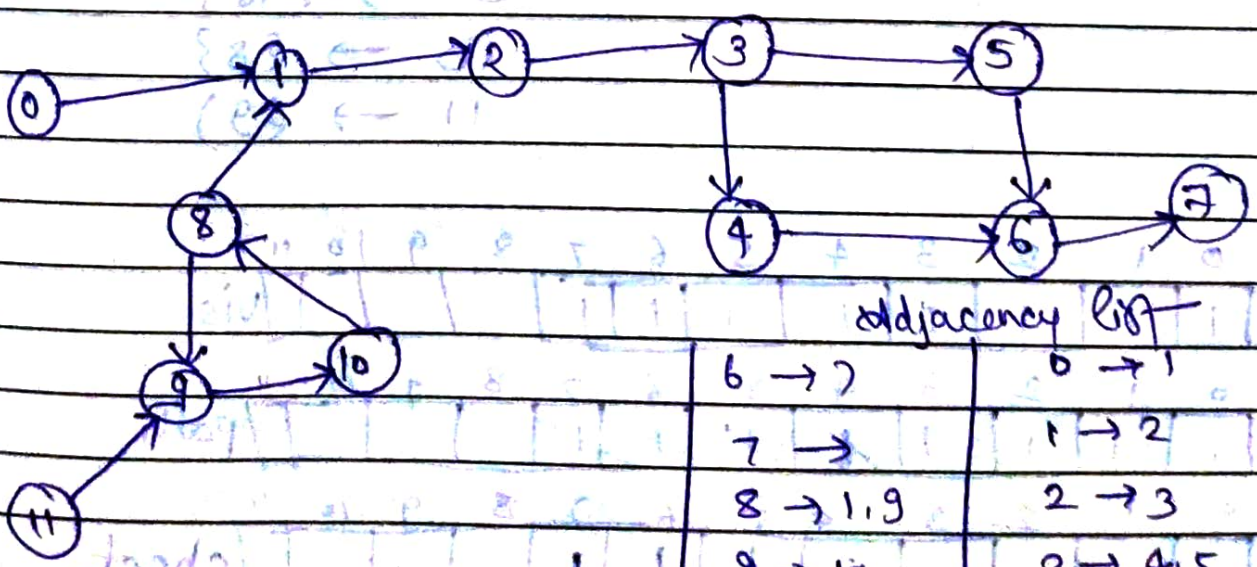
* any one who is a part of cycle. X
* Any one leads to a cycle X

⇒ Will Never Be Safe Nodes

Terminal Node → if there are No outgoing edges

Return वो Node जिसके सारे path terminal और भी Node पर जाते हैं

DRY RUN

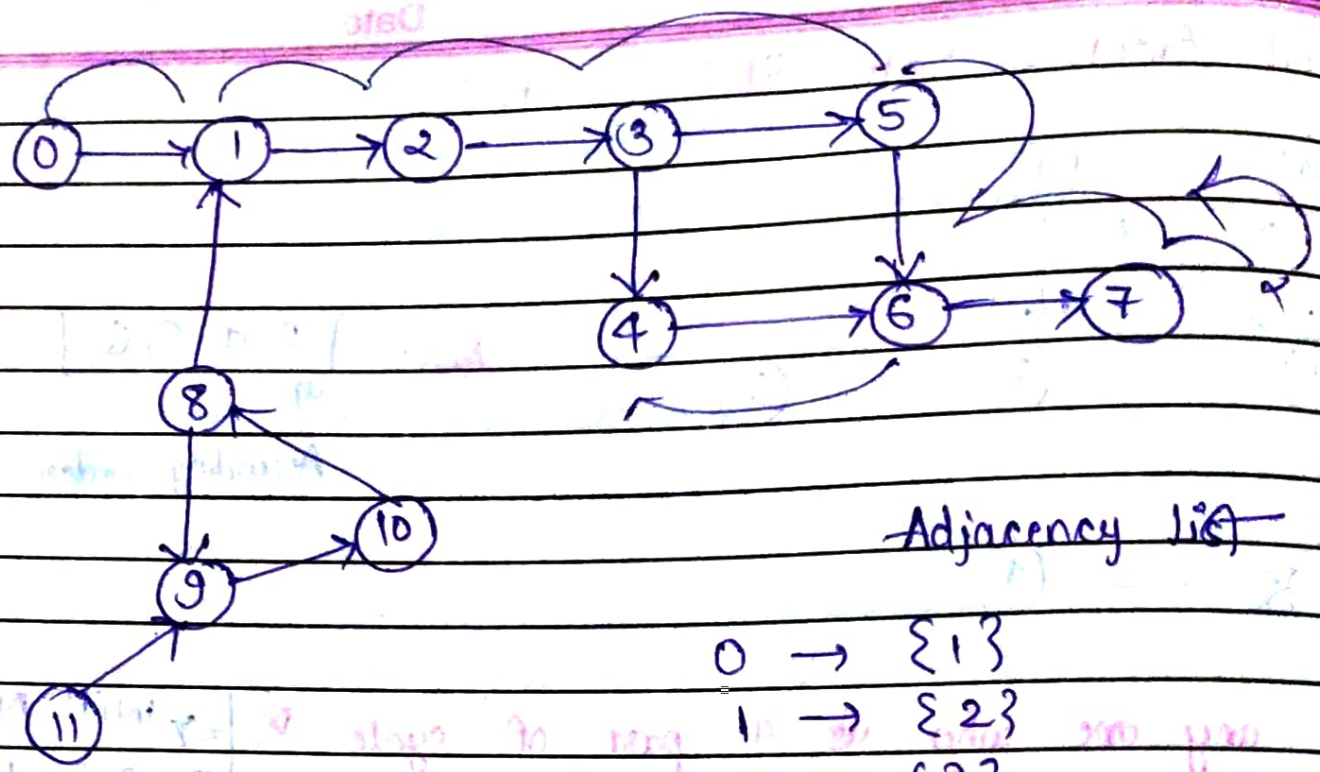


Adjacency list

6 → 7	0 → 1
7 →	1 → 2
8 → 1, 9	2 → 3
9 → 10	3 → 4, 5
10 → 8	4 → 6
11 → 9	5 → 6

#

Page No. :
Date :

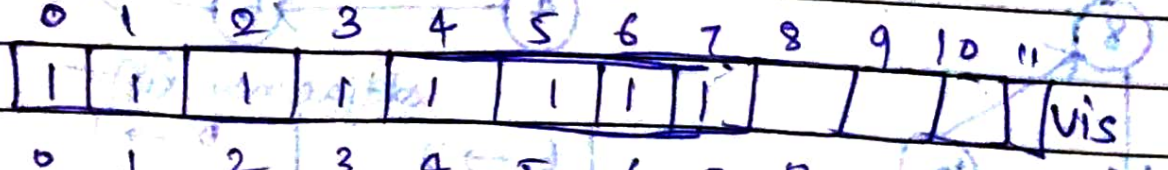


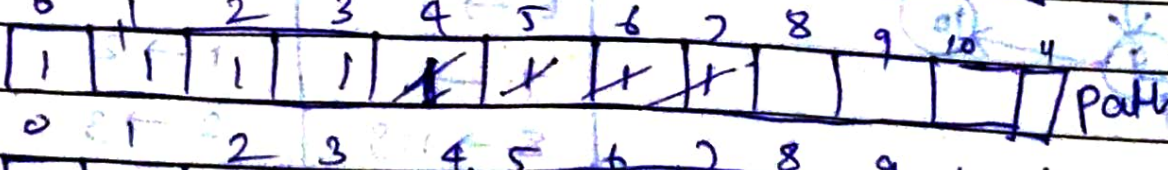
Adjacency list

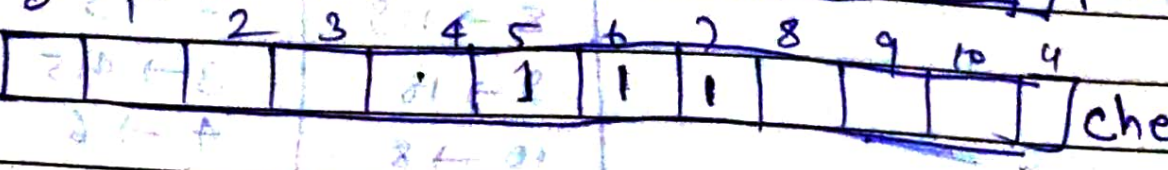
0 → {1}
1 → {2}
2 → {3}
3 → {4, 5}
4 → {6}
5 → {6}
6 → {7}
7 → {}
8 → {1, 9}
9 → {10}
10 → {8}
11 → {9}

(1) Start from Node 0

(3) → 5







Page No. :
Date :

(2) (3) → 4 ~~6~~ call

6 already visited
but Not pathVisited
No Need to call Visited already.

(8)

1	1	1	1	1	1	1	1	0	0	0	0	vis
X	1	X	X	X	0	0	0	0	0	0	0	pathVis
					1	1	1					check
0	1	2	3	4	5	6	7	8	9	10	11	

Now (1) → ~~8~~

1, 2, 3, 4, 5, 6, 7 already visited
No Need to call

(4) (8) → ~~2~~ call (8) → (9) → (10)

1	0	1		vis
1	1	1		pathVis
				Check
8	9	10	11	

10 call करेगा 8 को
already visited & pathVisited ~~8~~
पर true return

(5) (11) → (9) → (10) → (8)

9 ~~6~~ call already vis pathVis true return

6

Page No. :
Date :

pub // Code ⇒ {As Cycle detection in Directed Graph with some modification}

```
private boolean dfsCheck(int node,
        List<List<Integer>> adj,
        int vis[], int pathVis[], int check[]){

    vis[node] = 1;
    pathVis[node] = 1;
    check[node] = 0;

    for(int it : adj.get(node)){
        if(vis[it] == 0){
            if(dfsCheck(it, adj, vis, pathVis, check)){
                return true;
            }
        }
        else if(pathVis[it] == 1){
            return true;
        }
    }

    check[node] = 1;
    pathVis[node] = 0;

    return false;
}
}
pri
```